

```

# =====
# CyberNetX Paper 2 - 2026 DINOv2 LABEL-QUALITY AUDIT (one file)
# Modern stack: DINOv2 embeddings + semantic dedup + label-error detection
# Run as ONE cell in Google Colab (Drive must already be mounted).
# =====
import subprocess, sys
subprocess.run([sys.executable, "-m", "pip", "install", "-q", "cleanlab"])

import os, glob, numpy as np, pandas as pd, torch
from PIL import Image
import torchvision.transforms as T
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_predict
from cleanlab.filter import find_label_issues

base = "/content/drive/MyDrive/cyberbullying_dataset/"

# ---- 1. gather images ----
def gather(split):
    items = []
    sd = os.path.join(base, split)
    if not os.path.isdir(sd):
        return items
    for cls in sorted(os.listdir(sd)):
        cdir = os.path.join(sd, cls)
        if not os.path.isdir(cdir):
            continue
        for p in glob.glob(os.path.join(cdir, "*")):
            items.append({"path": p, "class_name": cls, "split": split})
    return items

data = gather("train") + gather("val")
classes = sorted(set(d["class_name"] for d in data))
c2i = {c: i for i, c in enumerate(classes)}
for d in data:
    d["label"] = c2i[d["class_name"]]
print(f"Total {len(data)} | classes {c2i}")

# ---- 2. load DINOv2 (foundation model, frozen) ----
device = "cuda" if torch.cuda.is_available() else "cpu"
dino = torch.hub.load('facebookresearch/dinov2', 'dinov2_vitb14').eval().to(device)
tf = T.Compose([
    T.Resize((224, 224)),
    T.ToTensor(),
    T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])

# ---- 3. extract DINOv2 embeddings (PIL loader, batched) ----
embs, y, paths, errors = [], [], [], []
batch_imgs, batch_meta = [], []

def flush():
    if not batch_imgs:
        return
    x = torch.stack(batch_imgs).to(device)
    with torch.no_grad():
        f = dino(x).cpu().numpy() # CLS embedding [B, 768]
    for k, (lab, pth) in enumerate(batch_meta):
        embs.append(f[k]); y.append(lab); paths.append(pth)
    batch_imgs.clear(); batch_meta.clear()

for d in data:
    try:
        im = Image.open(d["path"]).convert("RGB")
        batch_imgs.append(tf(im)); batch_meta.append((d["label"], d["path"]))
        if len(batch_imgs) == 32:
            flush()
    except Exception as e:
        errors.append((d["path"], str(e)))
flush()

X = np.array(embs); y = np.array(y)
print(f"Embedded {len(y)} images | errors {len(errors)} | shape {X.shape}")
if errors[:3]:
    print("Sample errors:", errors[:3])

# ---- 4. semantic duplicate + leakage detection (DINOv2 cosine) ----
Xn = X / (np.linalg.norm(X, axis=1, keepdims=True) + 1e-9)
sims = Xn @ Xn.T
np.fill_diagonal(sims, 0)
path_to_split = {d["path"]: d["split"] for d in data}
splits = [path_to_split[p] for p in paths]
dup, leak, dup_pairs = 0, 0, []
THR = 0.90
for i in range(len(paths)):
    for j in range(i + 1, len(paths)):
        if sims[i, j] >= THR:
            dup += 1
            dup_pairs.append((paths[i], paths[j], float(sims[i, j])))

```

```

        if splits[i] != splits[j]:
            leak += 1
    print(f"Semantic near-duplicate pairs (>{THR}): {dup} | train/val leakage: {leak}")
    pd.DataFrame(dup_pairs, columns=["image_a", "image_b", "similarity"]).to_csv(
        "duplicate_pairs_dinov2.csv", index=False)

# ---- 5. label-error detection on DINOv2 features ----
probs = cross_val_predict(LogisticRegression(max_iter=3000), X, y,
                           cv=5, method="predict_proba")
issues = find_label_issues(labels=y, pred_probs=probs,
                           return_indices_ranked_by="self_confidence")
print(f"\nLikely MISLABELLED images: {len(issues)} of {len(y)}")

pd.DataFrame({"path": [paths[i] for i in issues],
              "given_label": [classes[y[i]] for i in issues],
              "confidence_wrong": [1 - probs[i, y[i]] for i in issues]})
    .to_csv("label_issues_dinov2.csv", index=False)

print("\n===== 2026 AUDIT SUMMARY =====")
print(f"Images embedded (DINOv2)   : {len(y)}")
print(f"Semantic duplicate pairs    : {dup}")
print(f"Train/Val leakage pairs      : {leak}")
print(f"Likely mislabelled images    : {len(issues)}")
print(f"Saved: label_issues_dinov2.csv, duplicate_pairs_dinov2.csv")
print("=====")

```

Total 542 | classes {'bully': 0, 'no_bully': 1}
 Downloading: <https://github.com/facebookresearch/dinov2/zipball/main> to /root/.cache/torch/hub/main.zip
 Downloading: https://dl.fbaipublicfiles.com/dinov2/dinov2_vitb14/dinov2_vitb14_pretrain.pth to /root/.cache/torch/hub/checkpoints/dinov2_100%|██████████| 330M/330M [00:00<00:00, 349MB/s]
 Embedded 542 images | errors 0 | shape (542, 768)
 Semantic near-duplicate pairs (>0.9): 80 | train/val leakage: 32

Likely MISLABELLED images: 13 of 542

```

===== 2026 AUDIT SUMMARY =====
Images embedded (DINOv2)   : 542
Semantic duplicate pairs    : 80
Train/Val leakage pairs    : 32
Likely mislabelled images  : 13
Saved: label_issues_dinov2.csv, duplicate_pairs_dinov2.csv
=====

```

```

# =====
# CyberNetX Paper 2 - BUILD CLEAN DATASET (one file)
# Removes DINOv2 semantic duplicates + train/val leakage,
# drops likely-mislabelled images, makes a proper 70/15/15
# stratified train/val/test split, and writes a clean folder.
# Run as ONE cell in Google Colab (Drive mounted, GPU on).
# Then point your existing CyberNetX training code at:
#   /content/drive/MyDrive/cyberbullying_dataset_clean/
# =====

import subprocess, sys
subprocess.run([sys.executable, "-m", "pip", "install", "-q", "cleanlab"])

import os, glob, shutil, numpy as np, torch
from PIL import Image
import torchvision.transforms as T
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_predict, train_test_split
from cleanlab.filter import find_label_issues
from scipy.sparse.csgraph import connected_components
from scipy.sparse import csr_matrix

base = "/content/drive/MyDrive/cyberbullying_dataset/"
out = "/content/drive/MyDrive/cyberbullying_dataset_clean/"

def gather(split):
    items = []
    sd = os.path.join(base, split)
    if not os.path.isdir(sd):
        return items
    for cls in sorted(os.listdir(sd)):
        cdir = os.path.join(sd, cls)
        if not os.path.isdir(cdir):
            continue
        for p in glob.glob(os.path.join(cdir, "*")):
            items.append({"path": p, "class_name": cls})
    return items

data = gather("train") + gather("val")
classes = sorted(set(d["class_name"] for d in data))
c2i = {c: i for i, c in enumerate(classes)}
for d in data:
    d["label"] = c2i[d["class_name"]]

device = "cuda" if torch.cuda.is_available() else "cpu"
dino = torch.hub.load('facebookresearch/dinov2', 'dinov2_vitb14').eval().to(device)
tf = T.Compose([T.Resize((224, 224)), T.ToTensor(),
                T.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])])

embs, y, paths, bi, bm = [], [], [], [], []

```

```

def flush():
    if not bi:
        return
    x = torch.stack(bi).to(device)
    with torch.no_grad():
        f = dino(x).cpu().numpy()
    for k, (lab, pth) in enumerate(bm):
        embs.append(f[k]); y.append(lab); paths.append(pth)
    bi.clear(); bm.clear()

for d in data:
    try:
        im = Image.open(d["path"]).convert("RGB")
        bi.append(tf(im)); bm.append((d["label"], d["path"]))
        if len(bi) == 32:
            flush()
    except Exception:
        pass
flush()

X = np.array(embs); y = np.array(y); paths = np.array(paths)

# --- likely mislabels (confident learning on DINOv2 features) ---
probs = cross_val_predict(LogisticRegression(max_iter=3000), X, y,
                          cv=5, method="predict_proba")
issues = set(find_label_issues(labels=y, pred_probs=probs,
                              return_indices_ranked_by="self_confidence").tolist())

# --- duplicate clusters: keep ONE representative each (removes dups + leakage) ---
Xn = X / ((np.linalg.norm(X, axis=1, keepdims=True) + 1e-9)
sims = Xn @ Xn.T
A = (sims >= 0.90).astype(int); np.fill_diagonal(A, 0)
_, cc = connected_components(csr_matrix(A), directed=False)
keep, seen = [], set()
for i in range(len(paths)):
    if cc[i] not in seen:
        seen.add(cc[i]); keep.append(i)
keep = set(keep)

final = np.array([i for i in range(len(paths)) if i in keep and i not in issues])
print(f"Original {len(paths)} | removed dup {len(paths)-len(keep)} | "
      f"| removed mislabel {len(issues)} | clean {len(final)}")

# --- stratified 70/15/15 split ---
yf = y[final]
tr, tmp = train_test_split(final, test_size=0.30, stratify=yf, random_state=42)
va, te = train_test_split(tmp, test_size=0.50, stratify=y[tmp], random_state=42)

# --- write clean folders ---
if os.path.exists(out):
    shutil.rmtree(out)
for split, idxs in [("train", tr), ("val", va), ("test", te)]:
    for i in idxs:
        dst = os.path.join(out, split, classes[y[i]])
        os.makedirs(dst, exist_ok=True)
        shutil.copy(paths[i], os.path.join(dst, os.path.basename(paths[i])))

print("\nClean dataset ->", out)
for split, idxs in [("train", tr), ("val", va), ("test", te)]:
    print(f" {split}: {len(idxs)} | "
          f" (bully={int(sum(y[idxs]==0))}, no_bully={int(sum(y[idxs]==1))})")

```

Using cache found in /root/.cache/torch/hub/facebookresearch_dinov2_main
 Original 542 | removed dup 57 | removed mislabel 13 | clean 472

Clean dataset -> /content/drive/MyDrive/cyberbullying_dataset_clean/
 train: 330 (bully=152, no_bully=178)
 val: 71 (bully=32, no_bully=39)
 test: 71 (bully=33, no_bully=38)

```

# =====
# CyberNetX Paper 2 - TRAIN ON CLEAN DATA + HONEST TEST EVAL
# Faithful rebuild of the 3-CNN CyberNetX ensemble
# (ResNet50 + InceptionResNetV2 + Xception, frozen, fused),
# with CORRECT per-backbone preprocessing.
# Trains on clean/train, monitors clean/val, reports the
# honest number on the untouched clean/test set.
# Run as ONE cell in Google Colab (GPU on).
# =====
import numpy as np, tensorflow as tf
from tensorflow.keras import layers, Model, Input
from tensorflow.keras.applications import ResNet50, InceptionResNetV2, Xception
from tensorflow.keras.applications.resnet50 import preprocess_input as pre_res
from tensorflow.keras.applications.inception_resnet_v2 import preprocess_input as pre_inc
from tensorflow.keras.applications.xception import preprocess_input as pre_xcep
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

clean = "/content/drive/MyDrive/cyberbullying_dataset_clean/"
IMG, BATCH = (224, 224), 32 # if out-of-memory, set BATCH = 16

def load(split, shuffle):
    return tf.keras.utils.image_dataset_from_directory(

```

```

clean + split, image_size=IMG, batch_size=BATCH,
label_mode="int", class_names=["bully", "no_bully"], shuffle=shuffle)

train_ds = load("train", True).prefetch(tf.data.AUTOTUNE)
val_ds   = load("val", False).prefetch(tf.data.AUTOTUNE)
test_ds  = load("test", False).prefetch(tf.data.AUTOTUNE)

aug = tf.keras.Sequential([layers.RandomFlip("horizontal"),
                           layers.RandomRotation(0.1),
                           layers.RandomZoom(0.1)])

def make_backbone(net, name):
    m = net(include_top=False, pooling="avg", weights="imagenet")
    m.trainable = False
    m._name = name
    return m

res = make_backbone(ResNet50, "resnet50")
inc = make_backbone(InceptionResNetV2, "incresv2")
xcp = make_backbone(Xception, "xception")

inp = Input((224, 224, 3))
a = aug(inp)
fr = res(layers.Lambda(pre_res)(a))
fi = inc(layers.Lambda(pre_inc)(a))
fx = xcp(layers.Lambda(pre_xcep)(a))
z = layers.Concatenate()([fr, fi, fx])
z = layers.BatchNormalization()(z)
z = layers.Dense(512, activation="relu")(z)
z = layers.Dropout(0.5)(z)
z = layers.Dense(256, activation="relu")(z)
z = layers.Dropout(0.5)(z)
out = layers.Dense(1, activation="sigmoid")(z)
model = Model(inp, out)

model.compile(optimizer=tf.keras.optimizers.Adam(1e-4),
              loss="binary_crossentropy", metrics=["accuracy"])

cb = [tf.keras.callbacks.EarlyStopping(patience=8, restore_best_weights=True,
                                       monitor="val_loss")]
model.fit(train_ds, validation_data=val_ds, epochs=40, callbacks=cb, verbose=1)

# ----- HONEST evaluation on the untouched TEST set -----
y_true, y_prob = [], []
for xb, yb in test_ds:
    y_prob.extend(model.predict(xb, verbose=0).ravel())
    y_true.extend(yb.numpy())
y_true, y_prob = np.array(y_true), np.array(y_prob)
y_pred = (y_prob >= 0.5).astype(int)

print("\n===== HONEST TEST-SET RESULTS (clean data) =====")
print("Test Accuracy:", round(accuracy_score(y_true, y_pred) * 100, 2), "%")
print("\nConfusion matrix [rows = true bully, no_bully]:")
print(confusion_matrix(y_true, y_pred))
print("\n", classification_report(y_true, y_pred,
                                target_names=["bully", "no_bully"], digits=3))

Found 320 files belonging to 2 classes.
Found 71 files belonging to 2 classes.
Found 70 files belonging to 2 classes.
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/inception\_resnet\_v2/inception\_resnet\_v2\_weights\_tf\_dim\_ordering\_tf\_kernels\_219055592/219055592 1s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/xception/xception\_weights\_tf\_dim\_ordering\_tf\_kernels\_83683744/83683744 0s 0us/step
Epoch 1/40
10/10 ----- 56s 2s/step - accuracy: 0.6156 - loss: 0.7566 - val_accuracy: 0.9014 - val_loss: 0.3068
Epoch 2/40
10/10 ----- 6s 657ms/step - accuracy: 0.8781 - loss: 0.2860 - val_accuracy: 0.9296 - val_loss: 0.1951
Epoch 3/40
10/10 ----- 7s 679ms/step - accuracy: 0.9344 - loss: 0.1899 - val_accuracy: 0.9296 - val_loss: 0.1645
Epoch 4/40
10/10 ----- 7s 674ms/step - accuracy: 0.9281 - loss: 0.1935 - val_accuracy: 0.9296 - val_loss: 0.1417
Epoch 5/40
10/10 ----- 7s 687ms/step - accuracy: 0.9438 - loss: 0.1392 - val_accuracy: 0.9577 - val_loss: 0.1276
Epoch 6/40
10/10 ----- 7s 718ms/step - accuracy: 0.9469 - loss: 0.1234 - val_accuracy: 0.9577 - val_loss: 0.1211
Epoch 7/40
10/10 ----- 7s 689ms/step - accuracy: 0.9750 - loss: 0.0761 - val_accuracy: 0.9577 - val_loss: 0.1167
Epoch 8/40
10/10 ----- 10s 696ms/step - accuracy: 0.9781 - loss: 0.0737 - val_accuracy: 0.9577 - val_loss: 0.1134
Epoch 9/40
10/10 ----- 10s 681ms/step - accuracy: 0.9656 - loss: 0.1142 - val_accuracy: 0.9577 - val_loss: 0.1087
Epoch 10/40
10/10 ----- 7s 705ms/step - accuracy: 0.9594 - loss: 0.1109 - val_accuracy: 0.9577 - val_loss: 0.1070
Epoch 11/40
10/10 ----- 7s 682ms/step - accuracy: 0.9688 - loss: 0.0857 - val_accuracy: 0.9437 - val_loss: 0.1056
Epoch 12/40
10/10 ----- 7s 697ms/step - accuracy: 0.9781 - loss: 0.0594 - val_accuracy: 0.9577 - val_loss: 0.1053
Epoch 13/40
10/10 ----- 9s 599ms/step - accuracy: 0.9812 - loss: 0.0633 - val_accuracy: 0.9577 - val_loss: 0.1069
Epoch 14/40
10/10 ----- 7s 684ms/step - accuracy: 0.9719 - loss: 0.0533 - val_accuracy: 0.9577 - val_loss: 0.1046
Epoch 15/40
10/10 ----- 7s 683ms/step - accuracy: 0.9844 - loss: 0.0424 - val_accuracy: 0.9577 - val_loss: 0.1021
Epoch 16/40

```

```

10/10 ----- 7s 690ms/step - accuracy: 0.9812 - loss: 0.0488 - val_accuracy: 0.9577 - val_loss: 0.1009
Epoch 17/40
10/10 ----- 7s 680ms/step - accuracy: 0.9844 - loss: 0.0688 - val_accuracy: 0.9577 - val_loss: 0.0983
Epoch 18/40
10/10 ----- 7s 691ms/step - accuracy: 0.9688 - loss: 0.0459 - val_accuracy: 0.9577 - val_loss: 0.0959
Epoch 19/40
10/10 ----- 10s 611ms/step - accuracy: 0.9750 - loss: 0.0547 - val_accuracy: 0.9437 - val_loss: 0.0988
Epoch 20/40
10/10 ----- 6s 607ms/step - accuracy: 0.9906 - loss: 0.0351 - val_accuracy: 0.9437 - val_loss: 0.1013
Epoch 21/40
10/10 ----- 6s 608ms/step - accuracy: 0.9750 - loss: 0.0454 - val_accuracy: 0.9437 - val_loss: 0.1012
Epoch 22/40
10/10 ----- 6s 625ms/step - accuracy: 0.9875 - loss: 0.0466 - val_accuracy: 0.9437 - val_loss: 0.1034
Epoch 23/40
10/10 ----- 6s 607ms/step - accuracy: 0.9875 - loss: 0.0388 - val_accuracy: 0.9437 - val_loss: 0.1063
Epoch 24/40
10/10 ----- 6s 627ms/step - accuracy: 0.9844 - loss: 0.0488 - val_accuracy: 0.9577 - val_loss: 0.1140
Epoch 25/40
10/10 ----- 6s 604ms/step - accuracy: 0.9937 - loss: 0.0201 - val_accuracy: 0.9577 - val_loss: 0.1123

```

```

from google.colab import files
files.upload() # <-- a button appears; select your kaggle.json

!mkdir -p ~/.kaggle && cp kaggle.json ~/.kaggle/ && chmod 600 ~/.kaggle/kaggle.json
!pip install -q kaggle

# download the Hateful Memes dataset (Kaggle mirror) into Colab
!kaggle datasets download -d williamberrios/hateful-memes
!unzip -q hateful-memes.zip -d /content/hateful_memes
print("Done -> /content/hateful_memes")

```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving kaggle-end-to-end-training.ipynb to kaggle-end-to-end-training.ipynb

cp: cannot stat 'kaggle.json': No such file or directory

Warning: Looks like you're using an outdated `kaggle` version (installed: 2.0.2), please consider upgrading to the latest version (2.2.2)

Dataset URL: <https://www.kaggle.com/datasets/williamberrios/hateful-memes>

License(s): unknown

Downloading hateful-memes.zip to /content

100% 3.94G/3.94G [02:00<00:00, 35.1MB/s]

Done -> /content/hateful_memes

```

import os, json, glob

root = "/content/hateful_memes"

print("=== TOP-LEVEL STRUCTURE ===")
for r, dirs, fs in os.walk(root):
    depth = r.count("/") - root.count("/")
    if depth <= 1:
        print(r, "->", len(dirs), "folders,", len(fs), "files")

print("\n=== JSONL FILES ===")
for f in glob.glob(root + "/*.jsonl", recursive=True):
    print(f)

print("\n=== SAMPLE LINE ===")
jsonls = glob.glob(root + "/*.jsonl", recursive=True)
if jsonls:
    with open(jsonls[0]) as fh:
        print("From:", jsonls[0])
        print(fh.readline().strip())

print("\n=== IMAGE COUNT ===")
imgs = glob.glob(root + "/*.png", recursive=True) + glob.glob(root + "/*.jpg", recursive=True)
print("images found:", len(imgs))
if imgs:
    print("example:", imgs[0])

```

```

=== TOP-LEVEL STRUCTURE ===
/content/hateful_memes -> 1 folders, 0 files
/content/hateful_memes/hateful_memes -> 1 folders, 7 files

=== JSONL FILES ===
/content/hateful_memes/hateful_memes/test_seen.jsonl
/content/hateful_memes/hateful_memes/train.jsonl
/content/hateful_memes/hateful_memes/test_unseen.jsonl
/content/hateful_memes/hateful_memes/dev_unseen.jsonl
/content/hateful_memes/hateful_memes/dev_seen.jsonl

=== SAMPLE LINE ===
From: /content/hateful_memes/hateful_memes/test_seen.jsonl
{"id": "16395", "img": "img/16395.png", "label": 1, "text": "handjobs sold seperately"}

=== IMAGE COUNT ===
images found: 12140
example: /content/hateful_memes/hateful_memes/img/50273.png

```

```

# =====
# CyberNetX Paper 2 - CROSS-DATASET COLLAPSE TEST (full, robust)
# Mounts Drive, auto-(re)downloads Hateful Memes, trains on

```

```

# CLEAN stock data, tests on REAL memes. Saves kaggle key to
# Drive so you only upload it once, ever.
# Run as ONE cell in Google Colab (GPU on).
# =====
from google.colab import drive
drive.mount('/content/drive')
import os, json, numpy as np, tensorflow as tf
from PIL import Image
from tensorflow.keras import layers, Model, Input
from tensorflow.keras.applications import ResNet50, InceptionResNetV2, Xception
from tensorflow.keras.applications.resnet50 import preprocess_input as pre_res
from tensorflow.keras.applications.inception_resnet_v2 import preprocess_input as pre_inc
from tensorflow.keras.applications.xception import preprocess_input as pre_xcep
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score, recall_score

# ---- kaggle key (saved to Drive once, reused forever) ----
os.makedirs("/root/.kaggle", exist_ok=True)
drive_key = "/content/drive/MyDrive/kaggle.json"
if os.path.exists(drive_key):
    os.system(f"cp '{drive_key}' /root/.kaggle/kaggle.json")
    print("Kaggle key loaded from Drive.")
else:
    print(">> Upload kaggle.json now (file MUST be named kaggle.json):")
    from google.colab import files
    files.upload()
    os.system("cp kaggle.json /root/.kaggle/kaggle.json")
    os.system(f"cp kaggle.json '{drive_key}'")
    print("Kaggle key saved to Drive for next time.")
os.system("chmod 600 /root/.kaggle/kaggle.json")

# ---- get Hateful Memes if missing (~4GB, ~2 min) ----
hm = "/content/hateful_memes/hateful_memes/"
if not os.path.exists(hm):
    print("Downloading Hateful Memes... ~2 minutes.")
    os.system("pip install -q kaggle")
    os.system("kaggle datasets download -d williamberrios/hateful-memes -p /content")
    os.system("unzip -q /content/hateful-memes.zip -d /content/hateful_memes")
print("Hateful Memes ready:", os.path.exists(hm))

# ---- train CyberNetX on CLEAN stock data ----
clean = "/content/drive/MyDrive/cyberbullying_dataset_clean/"
def load(split, shuffle):
    return tf.keras.utils.image_dataset_from_directory(
        clean + split, image_size=(224, 224), batch_size=32,
        label_mode="int", class_names=["bully", "no_bully"], shuffle=shuffle)
train_ds = load("train", True).prefetch(tf.data.AUTOTUNE)
val_ds = load("val", False).prefetch(tf.data.AUTOTUNE)

aug = tf.keras.Sequential([layers.RandomFlip("horizontal"),
                           layers.RandomRotation(0.1), layers.RandomZoom(0.1)])

def bb(net, name):
    m = net(include_top=False, pooling="avg", weights="imagenet")
    m.trainable = False; m._name = name
    return m
res, inc, xcp = bb(ResNet50, "r"), bb(InceptionResNetV2, "i"), bb(Xception, "x")

inp = Input((224, 224, 3)); a = aug(inp)
z = layers.Concatenate()([res(layers.Lambda(pre_res)(a)),
                           inc(layers.Lambda(pre_inc)(a)),
                           xcp(layers.Lambda(pre_xcep)(a))])
z = layers.BatchNormalization()(z)
z = layers.Dense(512, activation="relu")(z); z = layers.Dropout(0.5)(z)
z = layers.Dense(256, activation="relu")(z); z = layers.Dropout(0.5)(z)
out = layers.Dense(1, activation="sigmoid")(z) # P(no_bully)
model = Model(inp, out)
model.compile(optimizer=tf.keras.optimizers.Adam(1e-4),
              loss="binary_crossentropy", metrics=["accuracy"])
cb = [tf.keras.callbacks.EarlyStopping(patience=8, restore_best_weights=True, monitor="val_loss")]
print("\nTraining on clean stock data...")
model.fit(train_ds, validation_data=val_ds, epochs=40, callbacks=cb, verbose=1)

# ---- test on REAL Hateful Memes ----
entries = []
for jf in ["dev_seen.jsonl", "test_seen.jsonl"]:
    p = os.path.join(hm, jf)
    if os.path.exists(p):
        entries += [json.loads(l) for l in open(p)]
print(f"\nReal Hateful Memes test samples: {len(entries)}")

def batches(items, bs=32):
    imgs, labs = [], []
    for e in items:
        try:
            im = Image.open(os.path.join(hm, e["img"])).convert("RGB").resize((224, 224))
            imgs.append(np.array(im, dtype="float32")); labs.append(e["label"])
            if len(imgs) == bs:
                yield np.stack(imgs), np.array(labs); imgs, labs = [], []
        except Exception:
            pass
    if imgs:
        yield np.stack(imgs), np.array(labs)

```

```

y_true, p_harm = [], []
for xb, yb in batches(entries):
    s = model.predict(xb, verbose=0).ravel() # P(no_bully)
    p_harm.extend(1 - s); y_true.extend(yb) # P(harmful); 1 = hateful
y_true, p_harm = np.array(y_true), np.array(p_harm)
y_pred = (p_harm >= 0.5).astype(int)

print("\n==== CROSS-DATASET RESULT: stock-trained model on REAL memes =====")
print("Real memes tested      :", len(y_true))
print("Accuracy on REAL data  :", round(accuracy_score(y_true, y_pred) * 100, 2), "%")
print("RECALL on real HARMFUL  :", round(recall_score(y_true, y_pred) * 100, 2), "% <-- key number")
print("\nConfusion matrix [rows = true not-harmful, harmful]:")
print(confusion_matrix(y_true, y_pred))
print("\n", classification_report(y_true, y_pred, target_names=["not-harmful", "harmful"], digits=3))

```

Mounted at /content/drive

>> Upload kaggle.json now (file MUST be named kaggle.json):

No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving kaggle-end-to-end-training.ipynb to kaggle-end-to-end-training.ipynb

Kaggle key saved to Drive for next time.

Downloading Hateful Memes... ~2 minutes.

Hateful Memes ready: True

Found 320 files belonging to 2 classes.

Found 71 files belonging to 2 classes.

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50_weights_tf_dim_ordering_tf_kernels_notop_94765736/94765736 6s 0us/step

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/inception_resnet_v2/inception_resnet_v2_weights_tf_dim_ordering_tf_kernels_notop_219055592/219055592 13s 0us/step

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/xception/xception_weights_tf_dim_ordering_tf_kernels_notop_83683744/83683744 5s 0us/step

Training on clean stock data...

Epoch 1/40

10/10 191s 16s/step - accuracy: 0.7031 - loss: 0.6172 - val_accuracy: 0.9437 - val_loss: 0.2273

Epoch 2/40

10/10 6s 650ms/step - accuracy: 0.8969 - loss: 0.2848 - val_accuracy: 0.9437 - val_loss: 0.1759

Epoch 3/40

10/10 6s 649ms/step - accuracy: 0.9219 - loss: 0.2025 - val_accuracy: 0.9437 - val_loss: 0.1540

Epoch 4/40

10/10 6s 649ms/step - accuracy: 0.9406 - loss: 0.1636 - val_accuracy: 0.9437 - val_loss: 0.1435

Epoch 5/40

10/10 6s 592ms/step - accuracy: 0.9250 - loss: 0.2147 - val_accuracy: 0.9437 - val_loss: 0.1466

Epoch 6/40

10/10 6s 567ms/step - accuracy: 0.9531 - loss: 0.1228 - val_accuracy: 0.9437 - val_loss: 0.1508

Epoch 7/40

10/10 6s 605ms/step - accuracy: 0.9563 - loss: 0.1130 - val_accuracy: 0.9437 - val_loss: 0.1496

Epoch 8/40

10/10 6s 573ms/step - accuracy: 0.9563 - loss: 0.0911 - val_accuracy: 0.9437 - val_loss: 0.1476

Epoch 9/40

10/10 7s 680ms/step - accuracy: 0.9531 - loss: 0.1042 - val_accuracy: 0.9437 - val_loss: 0.1396

Epoch 10/40

10/10 7s 656ms/step - accuracy: 0.9781 - loss: 0.0677 - val_accuracy: 0.9437 - val_loss: 0.1368

Epoch 11/40

10/10 6s 593ms/step - accuracy: 0.9656 - loss: 0.1167 - val_accuracy: 0.9437 - val_loss: 0.1383

Epoch 12/40

10/10 11s 658ms/step - accuracy: 0.9781 - loss: 0.0595 - val_accuracy: 0.9577 - val_loss: 0.1348

Epoch 13/40

10/10 6s 607ms/step - accuracy: 0.9781 - loss: 0.0625 - val_accuracy: 0.9577 - val_loss: 0.1350

Epoch 14/40

10/10 6s 592ms/step - accuracy: 0.9906 - loss: 0.0390 - val_accuracy: 0.9577 - val_loss: 0.1348

Epoch 15/40

10/10 6s 607ms/step - accuracy: 0.9875 - loss: 0.0501 - val_accuracy: 0.9577 - val_loss: 0.1387

Epoch 16/40

10/10 10s 591ms/step - accuracy: 0.9781 - loss: 0.0659 - val_accuracy: 0.9577 - val_loss: 0.1478

Epoch 17/40

10/10 6s 614ms/step - accuracy: 0.9875 - loss: 0.0325 - val_accuracy: 0.9577 - val_loss: 0.1535

Epoch 18/40

10/10 6s 599ms/step - accuracy: 0.9688 - loss: 0.0657 - val_accuracy: 0.9577 - val_loss: 0.1591

Epoch 19/40

10/10 6s 618ms/step - accuracy: 0.9781 - loss: 0.0545 - val_accuracy: 0.9577 - val_loss: 0.1588

Epoch 20/40

10/10 6s 604ms/step - accuracy: 0.9875 - loss: 0.0293 - val_accuracy: 0.9577 - val_loss: 0.1656

```

# =====
# CyberNetX Paper 2 - CROSS-DATASET COLLAPSE TEST (full, robust)
# Mounts Drive, auto-(re)downloads Hateful Memes, trains on
# CLEAN stock data, tests on REAL memes. Saves kaggle key to
# Drive so you only upload it once, ever.
# Run as ONE cell in Google Colab (GPU on).
# =====
from google.colab import drive
drive.mount('/content/drive')
import os, json, numpy as np, tensorflow as tf
from PIL import Image
from tensorflow.keras import layers, Model, Input
from tensorflow.keras.applications import ResNet50, InceptionResNetV2, Xception
from tensorflow.keras.applications.resnet50 import preprocess_input as pre_res
from tensorflow.keras.applications.inception_resnet_v2 import preprocess_input as pre_inc
from tensorflow.keras.applications.xception import preprocess_input as pre_xcep
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score, recall_score

# ---- kaggle key (saved to Drive once, reused forever) ----
os.makedirs("/root/.kaggle", exist_ok=True)
drive_key = "/content/drive/MyDrive/kaggle.json"
if os.path.exists(drive_key):

```

```

os.system(f"cp '{drive_key}' /root/.kaggle/kaggle.json")
print("Kaggle key loaded from Drive.")
else:
    print(">> Upload kaggle.json now (file MUST be named kaggle.json):")
    from google.colab import files
    files.upload()
    os.system("cp kaggle.json /root/.kaggle/kaggle.json")
    os.system(f"cp kaggle.json '{drive_key}'")
    print("Kaggle key saved to Drive for next time.")
os.system("chmod 600 /root/.kaggle/kaggle.json")

# ----- get Hateful Memes if missing (~4GB, ~2 min) -----
hm = "/content/hateful_memes/hateful_memes/"
if not os.path.exists(hm):
    print("Downloading Hateful Memes... ~2 minutes.")
    os.system("pip install -q kaggle")
    os.system("kaggle datasets download -d williamberrios/hateful-memes -p /content")
    os.system("unzip -q /content/hateful-memes.zip -d /content/hateful_memes")
print("Hateful Memes ready:", os.path.exists(hm))

# ----- train CyberNetX on CLEAN stock data -----
clean = "/content/drive/MyDrive/cyberbullying_dataset_clean/"
def load(split, shuffle):
    return tf.keras.utils.image_dataset_from_directory(
        clean + split, image_size=(224, 224), batch_size=32,
        label_mode="int", class_names=["bully", "no_bully"], shuffle=shuffle)
train_ds = load("train", True).prefetch(tf.data.AUTOTUNE)
val_ds = load("val", False).prefetch(tf.data.AUTOTUNE)

aug = tf.keras.Sequential([layers.RandomFlip("horizontal"),
                           layers.RandomRotation(0.1), layers.RandomZoom(0.1)])

def bb(net, name):
    m = net(include_top=False, pooling="avg", weights="imagenet")
    m.trainable = False; m._name = name
    return m
res, inc, xcp = bb(ResNet50, "r"), bb(InceptionResNetV2, "i"), bb(Xception, "x")

inp = Input((224, 224, 3)); a = aug(inp)
z = layers.Concatenate()([res(layers.Lambda(pre_res)(a)),
                           inc(layers.Lambda(pre_inc)(a)),
                           xcp(layers.Lambda(pre_xcp)(a))])
z = layers.BatchNormalization()(z)
z = layers.Dense(512, activation="relu")(z); z = layers.Dropout(0.5)(z)
z = layers.Dense(256, activation="relu")(z); z = layers.Dropout(0.5)(z)
out = layers.Dense(1, activation="sigmoid")(z) # P(no_bully)
model = Model(inp, out)
model.compile(optimizer=tf.keras.optimizers.Adam(1e-4),
              loss="binary_crossentropy", metrics=["accuracy"])
cb = [tf.keras.callbacks.EarlyStopping(patience=8, restore_best_weights=True, monitor="val_loss")]
print("\nTraining on clean stock data...")
model.fit(train_ds, validation_data=val_ds, epochs=40, callbacks=cb, verbose=1)

# ----- test on REAL Hateful Memes -----
entries = []
for jf in ["dev_seen.jsonl", "test_seen.jsonl"]:
    p = os.path.join(hm, jf)
    if os.path.exists(p):
        entries += [json.loads(l) for l in open(p)]
print(f"\nReal Hateful Memes test samples: {len(entries)}")

def batches(items, bs=32):
    imgs, labs = [], []
    for e in items:
        try:
            im = Image.open(os.path.join(hm, e["img"])).convert("RGB").resize((224, 224))
            imgs.append(np.array(im, dtype="float32")); labs.append(e["label"])
            if len(imgs) == bs:
                yield np.stack(imgs), np.array(labs); imgs, labs = [], []
        except Exception:
            pass
    if imgs:
        yield np.stack(imgs), np.array(labs)

y_true, p_harm = [], []
for xb, yb in batches(entries):
    s = model.predict(xb, verbose=0).ravel() # P(no_bully)
    p_harm.extend(1 - s); y_true.extend(yb) # P(harmful); 1 = hateful
y_true, p_harm = np.array(y_true), np.array(p_harm)
y_pred = (p_harm >= 0.5).astype(int)

print("\n===== CROSS-DATASET RESULT: stock-trained model on REAL memes =====")
print("Real memes tested      :", len(y_true))
print("Accuracy on REAL data   :", round(accuracy_score(y_true, y_pred) * 100, 2), "%")
print("RECALL on real HARMFUL   :", round(recall_score(y_true, y_pred) * 100, 2), "% <-- key number")
print("\nConfusion matrix [rows = true not-harmful, harmful]:")
print(confusion_matrix(y_true, y_pred))
print("\n", classification_report(y_true, y_pred, target_names=["not-harmful", "harmful"], digits=3))

```

```

Training on clean stock data...
Epoch 1/40
10/10  47s 2s/step - accuracy: 0.6531 - loss: 0.7346 - val_accuracy: 0.9437 - val_loss: 0.3213
Epoch 2/40
10/10  6s 648ms/step - accuracy: 0.8813 - loss: 0.2964 - val_accuracy: 0.9577 - val_loss: 0.1928
Epoch 3/40
10/10  7s 687ms/step - accuracy: 0.9250 - loss: 0.2113 - val_accuracy: 0.9577 - val_loss: 0.1576
Epoch 4/40
10/10  10s 655ms/step - accuracy: 0.9219 - loss: 0.2314 - val_accuracy: 0.9577 - val_loss: 0.1433
Epoch 5/40
10/10  7s 667ms/step - accuracy: 0.9469 - loss: 0.1261 - val_accuracy: 0.9577 - val_loss: 0.1334
Epoch 6/40
10/10  6s 658ms/step - accuracy: 0.9469 - loss: 0.1517 - val_accuracy: 0.9577 - val_loss: 0.1302
Epoch 7/40
10/10  7s 678ms/step - accuracy: 0.9438 - loss: 0.1185 - val_accuracy: 0.9577 - val_loss: 0.1264
Epoch 8/40
10/10  10s 690ms/step - accuracy: 0.9563 - loss: 0.1014 - val_accuracy: 0.9437 - val_loss: 0.1203
Epoch 9/40
10/10  7s 678ms/step - accuracy: 0.9688 - loss: 0.0920 - val_accuracy: 0.9577 - val_loss: 0.1124
Epoch 10/40
10/10  7s 691ms/step - accuracy: 0.9719 - loss: 0.0759 - val_accuracy: 0.9577 - val_loss: 0.1080
Epoch 11/40
10/10  10s 671ms/step - accuracy: 0.9750 - loss: 0.0609 - val_accuracy: 0.9577 - val_loss: 0.1050
Epoch 12/40
10/10  7s 686ms/step - accuracy: 0.9781 - loss: 0.0591 - val_accuracy: 0.9577 - val_loss: 0.1016
Epoch 13/40
10/10  7s 679ms/step - accuracy: 0.9875 - loss: 0.0444 - val_accuracy: 0.9577 - val_loss: 0.1000
Epoch 14/40
10/10  7s 700ms/step - accuracy: 0.9719 - loss: 0.0719 - val_accuracy: 0.9577 - val_loss: 0.0993
Epoch 15/40
10/10  7s 680ms/step - accuracy: 0.9750 - loss: 0.0644 - val_accuracy: 0.9577 - val_loss: 0.0973
Epoch 16/40
10/10  7s 702ms/step - accuracy: 0.9844 - loss: 0.0447 - val_accuracy: 0.9577 - val_loss: 0.0970
Epoch 17/40
10/10  6s 604ms/step - accuracy: 0.9875 - loss: 0.0312 - val_accuracy: 0.9577 - val_loss: 0.0973
Epoch 18/40
10/10  10s 608ms/step - accuracy: 0.9844 - loss: 0.0476 - val_accuracy: 0.9437 - val_loss: 0.0972
Epoch 19/40
10/10  6s 619ms/step - accuracy: 0.9812 - loss: 0.0381 - val_accuracy: 0.9577 - val_loss: 0.0994
Epoch 20/40
10/10  10s 600ms/step - accuracy: 0.9875 - loss: 0.0445 - val_accuracy: 0.9577 - val_loss: 0.0978
Epoch 21/40
10/10  7s 694ms/step - accuracy: 0.9969 - loss: 0.0272 - val_accuracy: 0.9577 - val_loss: 0.0953
Epoch 22/40
10/10  10s 708ms/step - accuracy: 0.9937 - loss: 0.0251 - val_accuracy: 0.9577 - val_loss: 0.0927
Epoch 23/40
10/10  10s 678ms/step - accuracy: 0.9906 - loss: 0.0312 - val_accuracy: 0.9577 - val_loss: 0.0916
Epoch 24/40
10/10  6s 631ms/step - accuracy: 0.9906 - loss: 0.0255 - val_accuracy: 0.9577 - val_loss: 0.0932
Epoch 25/40

```

```
# Step 1: Mount Google Drive (a popup/link will ask you to log in and allow access)
from google.colab import drive
drive.mount('/content/drive')

# Step 2: Now search for your dataset folders
import pathlib

drive_root = pathlib.Path("/content/drive/MyDrive")

print("Top-level folders in your Drive:")
for p in sorted(drive_root.iterdir()):
    if p.is_dir():
        print("-", p)

print("\nSearching for folders that look like your dataset (train/val/test/bully)...")
for p in drive_root.glob("**"):
    if p.is_dir() and p.name.lower() in ("train", "val", "test", "bully", "no_bully"):
        n_files = len(list(p.glob("*.jpg")))
        print(f"- {p}    {{n_files}} files")
```

```

Mounting /root/.cache/bazel/_bazel_root/stock-trained-model-on-REAL-memes =====
Replanning folders in your drive:
Accuracy of BSA of MyDrive/Andaz Dewali Funds
RECALL of BSA of MyDrive/BSA % <-- key number
- /content/drive/MyDrive/Bully_Data
Confusion of BSA of MyDrive/CSE-Department, (File responses)
[[39027011/drive/MyDrive/Certificate
{39027011/drive/MyDrive/Colab Notebooks
- /content/drive/MyDrive/College
- /content/drive/MyDrive/Competitions-addExamsuppoint (2020-2025) (File responses)
- /content/drive/MyDrive/CyberNetX-Tamil
notcharful/drive/MyDrive/CyberNetX-pr0j0205 763
- /content/drive/MyDrive/IQ0513 0.511 737
- /content/drive/MyDrive/Net Worth
- /content/drive/MyDrive/Ph.D. Regulation 1500
- /content/drive/MyDrive/PI0518 1500
weights/drive/MyDrive/ST0203PH CO20505,CUDDAL000
- /content/drive/MyDrive/anand
- /content/drive/MyDrive/audio cyberbully detection

```

```
- /content/drive/MyDrive/cyber audio data
- /content/drive/MyDrive/cyberbullying_dataset
- /content/drive/MyDrive/cyberbullying_dataset_clean
- /content/drive/MyDrive/plac erp
```

Searching for folders that look like your dataset (train/val/test/bully)...

```
- /content/drive/MyDrive/audio_cyberbully_detection/train (2 files)
- /content/drive/MyDrive/audio_cyberbully_detection/val (2 files)
- /content/drive/MyDrive/cyberbullying_dataset/train (2 files)
- /content/drive/MyDrive/cyberbullying_dataset/val (2 files)
- /content/drive/MyDrive/cyberbullying_dataset_clean/train (2 files)
- /content/drive/MyDrive/cyberbullying_dataset_clean/val (2 files)
- /content/drive/MyDrive/cyberbullying_dataset_clean/test (2 files)
- /content/drive/MyDrive/cyberbullying_dataset_clean/train/no_bully (178 files)
- /content/drive/MyDrive/cyberbullying_dataset_clean/train/bully (152 files)
- /content/drive/MyDrive/cyberbullying_dataset_clean/val/no_bully (39 files)
- /content/drive/MyDrive/cyberbullying_dataset_clean/val/bully (32 files)
- /content/drive/MyDrive/cyberbullying_dataset_clean/test/bully (33 files)
- /content/drive/MyDrive/cyberbullying_dataset_clean/test/no_bully (38 files)
- /content/drive/MyDrive/cyberbullying_dataset/train/no_bully (219 files)
- /content/drive/MyDrive/cyberbullying_dataset/train/bully (216 files)
- /content/drive/MyDrive/cyberbullying_dataset/val/no_bully (53 files)
- /content/drive/MyDrive/cyberbullying_dataset/val/bully (54 files)
- /content/drive/MyDrive/audio_cyberbully_detection/clean_final/train (2 files)
- /content/drive/MyDrive/audio_cyberbully_detection/clean_final/val (2 files)
```

```
import pathlib
```

```
print("Searching Drive for anything meme/hateful-related...")
for p in pathlib.Path("/content/drive/MyDrive").rglob("*"):
    if p.is_dir() and ("meme" in p.name.lower() or "hateful" in p.name.lower()):
        n_files = len(list(p.rglob("*.***")))
        print(f" - {p} (~{n_files} files)")
```

```
print("\nSearching /content/ (local, session-only storage) for the same...")
for p in pathlib.Path("/content").rglob("*"):
    if p.is_dir() and ("meme" in p.name.lower() or "hateful" in p.name.lower()):
        n_files = len(list(p.rglob("*.***")))
        print(f" - {p} (~{n_files} files)")
```

Searching Drive for anything meme/hateful-related...

Searching /content/ (local, session-only storage) for the same...

```
import pathlib
```

```
project_dir = pathlib.Path("/content/drive/MyDrive/CyberNetX_project")
```

```
print("Everything inside CyberNetX_project:")
for p in sorted(project_dir.rglob("*.***")):
    print(" -", p)
```

```
print("\nLooking for your saved kaggle.json credential...")
for p in pathlib.Path("/content/drive/MyDrive").rglob("kaggle.json"):
    print(" - found:", p)
```

Everything inside CyberNetX_project:

```
- /content/drive/MyDrive/CyberNetX_project/features
- /content/drive/MyDrive/CyberNetX_project/features/ablation_results.csv
- /content/drive/MyDrive/CyberNetX_project/features/cv_results.csv
- /content/drive/MyDrive/CyberNetX_project/features/cybernetx_v2_results.csv
- /content/drive/MyDrive/CyberNetX_project/features/feature_importance_plot.png
- /content/drive/MyDrive/CyberNetX_project/features/test_checkpoint.npz
- /content/drive/MyDrive/CyberNetX_project/features/test_results.csv
- /content/drive/MyDrive/CyberNetX_project/features/train_checkpoint.npz
```

Looking for your saved kaggle.json credential...

```
import pandas as pd
import numpy as np
```

```
base = "/content/drive/MyDrive/CyberNetX_project/features/"
```

```
for fname in ["ablation_results.csv", "cv_results.csv", "cybernetx_v2_results.csv", "test_results.csv"]:
    print(f"\n{' '*60}\n{fname}\n{' '*60}")
    try:
```

```
        df = pd.read_csv(base + fname)
        print("Columns:", list(df.columns))
        print("Shape:", df.shape)
        print(df.head(10).to_string())
```

```
    except Exception as e:
        print("Could not read:", e)
```

```
for fname in ["train_checkpoint.npz", "test_checkpoint.npz"]:
```

```
    print(f"\n{' '*60}\n{fname}\n{' '*60}")
    try:
        data = np.load(base + fname, allow_pickle=True)
        print("Keys inside:", list(data.keys()))
        for k in data.keys():
            arr = data[k]
            print(f" {k}: shape={getattr(arr, 'shape', 'n/a')}, dtype={getattr(arr, 'dtype', type(arr))}")
    except Exception as e:
        print("Could not read:", e)
```

```

=====
ablation_results.csv
=====
Columns: ['Feature set', 'Accuracy', 'Precision', 'Recall', 'F1-Score', 'AUC']
Shape: (3, 6)

```

	Feature set	Accuracy	Precision	Recall	F1-Score	AUC
0	Traditional features only	0.790	0.753	0.772	0.762	0.874
1	Deep embeddings only	0.787	0.757	0.754	0.756	0.870
2	Fused (full CyberNetX)	0.801	0.769	0.776	0.773	0.886

```

=====
cv_results.csv
=====
Columns: ['Algorithm', 'Accuracy', 'Precision', 'Recall', 'F1-Score', 'AUC']
Shape: (4, 6)

```

	Algorithm	Accuracy	Precision	Recall	F1-Score	AUC
0	SVM	0.788 ± 0.008	0.751 ± 0.009	0.764 ± 0.014	0.758 ± 0.010	0.861 ± 0.009
1	Naive Bayes	0.759 ± 0.006	0.755 ± 0.012	0.655 ± 0.014	0.701 ± 0.008	0.804 ± 0.009
2	CyberNetX	0.783 ± 0.010	0.758 ± 0.014	0.733 ± 0.008	0.745 ± 0.010	0.863 ± 0.008
3	XGBoost	0.789 ± 0.008	0.751 ± 0.011	0.766 ± 0.010	0.759 ± 0.009	0.872 ± 0.009

```

=====
cybernetx_v2_results.csv
=====
Columns: ['Algorithm', 'Accuracy', 'Precision', 'Recall', 'F1-Score', 'AUC']
Shape: (4, 6)

```

	Algorithm	Accuracy	Precision	Recall	F1-Score	AUC
0	SVM	0.801	0.767	0.781	0.774	0.876
1	Random Forest	0.798	0.761	0.783	0.772	0.882
2	XGBoost	0.799	0.764	0.780	0.772	0.885
3	CyberNetX v2	0.801	0.769	0.776	0.773	0.886

```

=====
test_results.csv
=====
Columns: ['Algorithm', 'Accuracy', 'Precision', 'Recall', 'F1-Score', 'AUC']
Shape: (4, 6)

```

	Algorithm	Accuracy	Precision	Recall	F1-Score	AUC
0	SVM	0.801	0.767	0.781	0.774	0.876
1	Naive Bayes	0.767	0.765	0.673	0.716	0.817
2	XGBoost	0.799	0.764	0.780	0.772	0.885
3	CyberNetX	0.794	0.765	0.761	0.763	0.879

```

=====
train_checkpoint.npz
=====
Keys inside: ['trad', 'deep', 'labels', 'languages']
trad: shape=(8128, 376), dtype=object
deep: shape=(8128, 768), dtype=object
labels: shape=(8128,), dtype=int64
languages: shape=(8128,), dtype=object
=====
test_checkpoint.npz
=====
Keys inside: ['trad', 'deep', 'labels', 'languages']
trad: shape=(3647, 376), dtype=object

```

```

import pathlib

print("Searching all your Colab notebooks for the one that matches THIS paper...")
print("(looking for mentions of InceptionResNetV2, Xception, or DINOv2 together)\n")

search_terms = ["InceptionResNetV2", "Xception", "DINOv2", "dinov2"]

for nb_path in pathlib.Path("/content/drive/MyDrive").rglob("*.ipynb"):
    try:
        text = nb_path.read_text(errors="ignore")
    except Exception:
        continue
    hits = [t for t in search_terms if t in text]
    if len(hits) >= 2: # needs at least 2 matching terms to be a strong candidate
        print(f"STRONG MATCH: {nb_path}")
        print(f"    contains: {hits}\n")
    elif len(hits) == 1:
        print(f"possible match: {nb_path} (contains: {hits})")

Searching all your Colab notebooks for the one that matches THIS paper...
(looking for mentions of InceptionResNetV2, Xception, or DINOv2 together)

STRONG MATCH: /content/drive/MyDrive/Colab Notebooks/Untitled4.ipynb
contains: ['InceptionResNetV2', 'Xception']

STRONG MATCH: /content/drive/MyDrive/Colab Notebooks/Untitled7.ipynb
contains: ['InceptionResNetV2', 'Xception']

possible match: /content/drive/MyDrive/Colab Notebooks/Paper 4.ipynb (contains: ['Xception'])
STRONG MATCH: /content/drive/MyDrive/Colab Notebooks/High Medium Low.ipynb
contains: ['InceptionResNetV2', 'Xception']

STRONG MATCH: /content/drive/MyDrive/Colab Notebooks/Paper 3.ipynb
contains: ['InceptionResNetV2', 'Xception']

STRONG MATCH: /content/drive/MyDrive/Colab Notebooks/LogiForestX Full Details.ipynb
contains: ['InceptionResNetV2', 'Xception']

STRONG MATCH: /content/drive/MyDrive/Colab Notebooks/CyberNetX - Image.ipynb

```

```
contains: ['InceptionResNetV2', 'Xception']

STRONG MATCH: /content/drive/MyDrive/Colab Notebooks/Untitled5.ipynb
contains: ['InceptionResNetV2', 'Xception']

STRONG MATCH: /content/drive/MyDrive/Colab Notebooks/cybernetx - updated.ipynb
contains: ['InceptionResNetV2', 'Xception']

STRONG MATCH: /content/drive/MyDrive/Colab Notebooks/CyberNetX - Up.ipynb
contains: ['InceptionResNetV2', 'Xception']

STRONG MATCH: /content/drive/MyDrive/Colab Notebooks/Image Preprocessing.ipynb
contains: ['InceptionResNetV2', 'Xception', 'DINOv2', 'dinov2']
```

```
import pathlib

print("Searching for the notebook(s) containing this paper's exact reported numbers...\n")

target_numbers = ["95.71", "51.87", "Hateful Memes", "hateful_memes", "cross-dataset", "cross_dataset"]

for nb_path in pathlib.Path("/content/drive/MyDrive").rglob("*.ipynb"):
    try:
        text = nb_path.read_text(errors="ignore")
    except Exception:
        continue
    hits = [t for t in target_numbers if t in text]
    if hits:
        print(f"MATCH: {nb_path}")
        print(f"    contains: {hits}\n")
```

Searching for the notebook(s) containing this paper's exact reported numbers...

```
MATCH: /content/drive/MyDrive/Colab Notebooks/Image Preprocessing.ipynb
contains: ['95.71', '51.87', 'Hateful Memes', 'hateful_memes']
```

```
# =====
# FIX 1 – Rebuild the clean dataset WITHOUT filename collisions
# Run this ONCE, in place of your original "CyberNetX Paper 2 -
# BUILD CLEAN DATASET" cell. Everything is identical to your
# original code EXCEPT the copy step now gives every file a
# unique name, and a real verification step at the end actually
# counts what's on disk instead of trusting the in-memory split
# sizes.
# =====
import subprocess, sys
subprocess.run([sys.executable, "-m", "pip", "install", "-q", "cleanlab"])

import os, glob, shutil, numpy as np, torch
from PIL import Image
import torchvision.transforms as T
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_predict, train_test_split
from cleanlab.filter import find_label_issues
from scipy.sparse.csgraph import connected_components
from scipy.sparse import csr_matrix

base = "/content/drive/MyDrive/cyberbullying_dataset/"
out = "/content/drive/MyDrive/cyberbullying_dataset_clean/"

def gather(split):
    items = []
    sd = os.path.join(base, split)
    if not os.path.isdir(sd):
        return items
    for cls in sorted(os.listdir(sd)):
        cdir = os.path.join(sd, cls)
        if not os.path.isdir(cdir):
            continue
        for p in glob.glob(os.path.join(cdir, "*")):
            items.append({"path": p, "class_name": cls})
    return items

data = gather("train") + gather("val")
classes = sorted(set(d["class_name"] for d in data))
c2i = {c: i for i, c in enumerate(classes)}
for d in data:
    d["label"] = c2i[d["class_name"]]

device = "cuda" if torch.cuda.is_available() else "cpu"
dino = torch.hub.load('facebookresearch/dinov2', 'dinov2_vitb14').eval().to(device)
tf = T.Compose([T.Resize((224, 224)), T.ToTensor(),
                T.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])])

embs, y, paths, bi, bm = [], [], [], [], []
def flush():
    if not bi:
        return
    x = torch.stack(bi).to(device)
    with torch.no_grad():
        f = dino(x).cpu().numpy()
```

```

    for k, (lab, pth) in enumerate(bm):
        embs.append(f[k]); y.append(lab); paths.append(pth)
    bi.clear(); bm.clear()

for d in data:
    try:
        im = Image.open(d["path"]).convert("RGB")
        bi.append(tf(im)); bm.append((d["label"], d["path"]))
        if len(bi) == 32:
            flush()
    except Exception:
        pass
flush()

X = np.array(embs); y = np.array(y); paths = np.array(paths)

# --- likely mislabels (confident learning on DINOv2 features) ---
probs = cross_val_predict(LogisticRegression(max_iter=3000), X, y,
                          cv=5, method="predict_proba")
issues = set(find_label_issues(labels=y, pred_probs=probs,
                               return_indices_ranked_by="self_confidence").tolist())

# --- duplicate clusters: keep ONE representative each (removes dups + leakage) ---
Xn = X / (np.linalg.norm(X, axis=1, keepdims=True) + 1e-9)
sims = Xn @ Xn.T
A = (sims >= 0.90).astype(int); np.fill_diagonal(A, 0)
_, cc = connected_components(csrf_matrix(A), directed=False)
keep, seen = [], set()
for i in range(len(paths)):
    if cc[i] not in seen:
        seen.add(cc[i]); keep.append(i)
keep = set(keep)

final = np.array([i for i in range(len(paths)) if i in keep and i not in issues])
print(f"Original {len(paths)} | removed dup {len(paths)-len(keep)} "
      f"| removed mislabel {len(issues)} | clean {len(final)}")

# --- stratified 70/15/15 split ---
yf = y[final]
tr, tmp = train_test_split(final, test_size=0.30, stratify=yf, random_state=42)
va, te = train_test_split(tmp, test_size=0.50, stratify=y[tmp], random_state=42)

# --- write clean folders, WITH UNIQUE FILENAMES this time ---
if os.path.exists(out):
    shutil.rmtree(out)
for split, idxs in [("train", tr), ("val", va), ("test", te)]:
    for i in idxs:
        dst = os.path.join(out, split, classes[y[i]])
        os.makedirs(dst, exist_ok=True)
        # THE FIX: prefix with the unique original index so no two
        # files can ever collide, even if their original names match.
        unique_name = f"{i:05d}_{os.path.basename(paths[i])}"
        shutil.copy(paths[i], os.path.join(dst, unique_name))

print("\nIntended split sizes (from in-memory indices):")
for split, idxs in [("train", tr), ("val", va), ("test", te)]:
    print(f" {split}: {len(idxs)} "
          f"(bully={int(sum(y[idxs]==0))}, no_bully={int(sum(y[idxs]==1))})")

# --- REAL VERIFICATION: count what's actually on disk ---
print("\nVERIFIED counts actually on disk (this is the number that matters):")
grand_total = 0
for split in ["train", "val", "test"]:
    split_total = 0
    for cls in classes:
        d = os.path.join(out, split, cls)
        n = len(os.listdir(d)) if os.path.isdir(d) else 0
        split_total += n
        print(f" {split}/{cls}: {n} files")
    grand_total += split_total
    print(f" {split} TOTAL: {split_total} files")
print(f"GRAND TOTAL: {grand_total} files (should be 472 if the fix worked)")

```

Downloading: <https://github.com/facebookresearch/dinov2/zipball/main> to /root/.cache/torch/hub/main.zip
 Downloading: https://dl.fbaipublicfiles.com/dinov2/dinov2_vitb14/dinov2_vitb14_pretrain.pth to /root/.cache/torch/hub/checkpoints/dinov2_100%
 330M/330M [00:01<00:00, 285MB/s]
 Original 542 | removed dup 57 | removed mislabel 13 | clean 472

Intended split sizes (from in-memory indices):
 train: 330 (bully=152, no_bully=178)
 val: 71 (bully=32, no_bully=39)
 test: 71 (bully=33, no_bully=38)

VERIFIED counts actually on disk (this is the number that matters):
 train/bully: 152 files
 train/no_bully: 178 files
 train TOTAL: 330 files
 val/bully: 32 files
 val/no_bully: 39 files
 val TOTAL: 71 files
 test/bully: 33 files
 test/no_bully: 38 files

```
test TOTAL: 71 files
GRAND TOTAL: 472 files (should be 472 if the fix worked)
```

```
# =====
# FIX 2 – Train CyberNetX on the CORRECTED clean data, across
# 3 fixed seeds, with full determinism. This uses your EXACT
# architecture and hyperparameters from "Image Preprocessing.ipynb"
# – nothing about the model or training config has changed, only:
# (a) the data now comes from the collision-fixed folder (Fix 1)
# (b) randomness is now fully pinned
# (c) it loops over 3 seeds and records every result
#
# Run FIX 1 first. Then run this cell.
# =====
import os, random
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, Model, Input
from tensorflow.keras.applications import ResNet50, InceptionResNetV2, Xception
from tensorflow.keras.applications.resnet50 import preprocess_input as pre_res
from tensorflow.keras.applications.inception_resnet_v2 import preprocess_input as pre_inc
from tensorflow.keras.applications.xception import preprocess_input as pre_xcep
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

def set_full_determinism(seed: int):
    os.environ["PYTHONHASHSEED"] = str(seed)
    os.environ["TF_DETERMINISTIC_OPS"] = "1"
    os.environ["TF_CUDNN_DETERMINISTIC"] = "1"
    random.seed(seed); np.random.seed(seed); tf.random.set_seed(seed)
    try:
        tf.config.experimental.enable_op_determinism()
    except Exception as e:
        print("Warning: enable_op_determinism() unavailable:", e)
        print(f"[reproducibility] All seeds pinned to {seed}.")

clean = "/content/drive/MyDrive/cyberbullying_dataset_clean/"
IMG, BATCH = (224, 224), 32
SEEDS = [42, 123, 2024]

def load(split, shuffle, seed):
    return tf.keras.utils.image_dataset_from_directory(
        clean + split, image_size=IMG, batch_size=BATCH,
        label_mode="int", class_names=["bully", "no_bully"],
        shuffle=shuffle, seed=seed if shuffle else None)

def build_model():
    aug = tf.keras.Sequential([layers.RandomFlip("horizontal"),
                               layers.RandomRotation(0.1),
                               layers.RandomZoom(0.1)])

    def bb(net, name):
        m = net(include_top=False, pooling="avg", weights="imagenet")
        m.trainable = False
        m._name = name
        return m

    res, inc, xcp = bb(ResNet50, "resnet50"), bb(InceptionResNetV2, "incresv2"), bb(Xception, "xception")
    inp = Input((224, 224, 3)); a = aug(inp)
    fr = res(layers.Lambda(pre_res))(a)
    fi = inc(layers.Lambda(pre_inc))(a)
    fx = xcp(layers.Lambda(pre_xcep))(a)
    z = layers.Concatenate()([fr, fi, fx])
    z = layers.BatchNormalization()(z)
    z = layers.Dense(512, activation="relu")(z)
    z = layers.Dropout(0.5)(z)
    z = layers.Dense(256, activation="relu")(z)
    z = layers.Dropout(0.5)(z)
    out = layers.Dense(1, activation="sigmoid")(z)
    model = Model(inp, out)
    model.compile(optimizer=tf.keras.optimizers.Adam(1e-4),
                  loss="binary_crossentropy", metrics=["accuracy"])
    return model

def evaluate(model, ds):
    y_true, y_prob = [], []
    for xb, yb in ds:
        y_prob.extend(model.predict(xb, verbose=0).ravel())
        y_true.extend(yb.numpy())
    y_true, y_prob = np.array(y_true), np.array(y_prob)
    y_pred = (y_prob >= 0.5).astype(int)
    acc = round(accuracy_score(y_true, y_pred) * 100, 2)
    cm = confusion_matrix(y_true, y_pred)
    report = classification_report(y_true, y_pred, target_names=["bully", "no_bully"],
                                   digits=3, output_dict=True)

    return acc, cm, report

all_results = []
trained_models = [] # keep these around if you want to reuse one for Grad-CAM afterward

for seed in SEEDS:
```

```

print(f"\n{' '*20} SEED {seed} {' '*20}")
set_full_determinism(seed)

train_ds = load("train", True, seed).prefetch(tf.data.AUTOTUNE)
val_ds   = load("val", False, seed).prefetch(tf.data.AUTOTUNE)
test_ds  = load("test", False, seed).prefetch(tf.data.AUTOTUNE)

model = build_model()
cb = [tf.keras.callbacks.EarlyStopping(patience=8, restore_best_weights=True, monitor="val_loss")]
model.fit(train_ds, validation_data=val_ds, epochs=40, callbacks=cb, verbose=1)

print("\n----- HONEST in-domain evaluation on the untouched TEST set -----")
in_acc, in_cm, in_report = evaluate(model, test_ds)
print(f"In-domain test accuracy: {in_acc}%")
print("Confusion matrix [rows = true bully, no_bully]:\n", in_cm)

all_results.append({"seed": seed, "in_domain_acc": in_acc,
                   "in_domain_cm": in_cm.tolist(), "in_domain_report": in_report})
trained_models.append(model)

# -----
# PASTE YOUR EXISTING HATEFUL MEMES LOADING + EVALUATION CODE
# HERE, unchanged (the part of your "CROSS-DATASET COLLAPSE
# TEST" cell that comes after training and produces "Real
# Hateful Memes test samples: 1500" and the confusion matrix).
# It already works correctly and doesn't need any fix – just
# call it here using this loop's `model`, and append its
# accuracy/confusion matrix to all_results[-1] the same way
# as the in-domain block above, e.g.:
#
# cross_acc, cross_cm, cross_report = your_existing_eval_function(model)
# all_results[-1]["cross_acc"] = cross_acc
# all_results[-1]["cross_cm"] = cross_cm
# -----

print("\n\n===== FINAL SUMMARY =====")
in_accs = [r["in_domain_acc"] for r in all_results]
print(f"In-domain accuracy across seeds: {in_accs}")
print(f" mean = {np.mean(in_accs):.2f}% std = {np.std(in_accs):.2f}%")
print("\nFull results (copy this whole block back to Claude):")
import json
print(json.dumps(all_results, indent=2, default=str))

```

```

===== SEED 42 =====
[reproducibility] All seeds pinned to 42.
Found 320 files belonging to 2 classes.
Found 71 files belonging to 2 classes.
Found 70 files belonging to 2 classes.
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50\_weights\_tf\_dim\_ordering\_tf\_kernels\_notr
94765736/94765736 ————— 6s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/inception\_resnet\_v2/inception\_resnet\_v2\_weights\_tf\_dim\_ordering\_tf\_kernels\_notr
219055592/219055592 ————— 11s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/xception/xception\_weights\_tf\_dim\_ordering\_tf\_kernels\_notr
83683744/83683744 ————— 5s 0us/step
Epoch 1/40
10/10 ————— 51s 2s/step - accuracy: 0.7094 - loss: 0.6445 - val_accuracy: 0.9296 - val_loss: 0.3344
Epoch 2/40
10/10 ————— 7s 685ms/step - accuracy: 0.8625 - loss: 0.3219 - val_accuracy: 0.9437 - val_loss: 0.2297
Epoch 3/40
10/10 ————— 7s 760ms/step - accuracy: 0.9062 - loss: 0.2264 - val_accuracy: 0.9437 - val_loss: 0.1848
Epoch 4/40
10/10 ————— 10s 697ms/step - accuracy: 0.9219 - loss: 0.1895 - val_accuracy: 0.9437 - val_loss: 0.1620
Epoch 5/40
10/10 ————— 10s 706ms/step - accuracy: 0.9500 - loss: 0.1502 - val_accuracy: 0.9296 - val_loss: 0.1588
Epoch 6/40
10/10 ————— 7s 731ms/step - accuracy: 0.9438 - loss: 0.1185 - val_accuracy: 0.9437 - val_loss: 0.1471
Epoch 7/40
10/10 ————— 7s 723ms/step - accuracy: 0.9438 - loss: 0.1201 - val_accuracy: 0.9718 - val_loss: 0.1306
Epoch 8/40
10/10 ————— 7s 736ms/step - accuracy: 0.9563 - loss: 0.1136 - val_accuracy: 0.9718 - val_loss: 0.1207
Epoch 9/40
10/10 ————— 7s 707ms/step - accuracy: 0.9656 - loss: 0.0742 - val_accuracy: 0.9718 - val_loss: 0.1190
Epoch 10/40
10/10 ————— 7s 718ms/step - accuracy: 0.9906 - loss: 0.0594 - val_accuracy: 0.9577 - val_loss: 0.1157
Epoch 11/40
10/10 ————— 6s 620ms/step - accuracy: 0.9750 - loss: 0.0757 - val_accuracy: 0.9577 - val_loss: 0.1185
Epoch 12/40
10/10 ————— 7s 676ms/step - accuracy: 0.9625 - loss: 0.0854 - val_accuracy: 0.9437 - val_loss: 0.1197
Epoch 13/40
10/10 ————— 6s 626ms/step - accuracy: 0.9719 - loss: 0.0639 - val_accuracy: 0.9437 - val_loss: 0.1159
Epoch 14/40
10/10 ————— 7s 727ms/step - accuracy: 0.9812 - loss: 0.0730 - val_accuracy: 0.9437 - val_loss: 0.1070
Epoch 15/40
10/10 ————— 7s 707ms/step - accuracy: 0.9781 - loss: 0.0493 - val_accuracy: 0.9437 - val_loss: 0.1041
Epoch 16/40
10/10 ————— 7s 710ms/step - accuracy: 0.9781 - loss: 0.0494 - val_accuracy: 0.9577 - val_loss: 0.1014
Epoch 17/40
10/10 ————— 7s 712ms/step - accuracy: 0.9781 - loss: 0.0487 - val_accuracy: 0.9577 - val_loss: 0.0998
Epoch 18/40
10/10 ————— 7s 715ms/step - accuracy: 0.9875 - loss: 0.0387 - val_accuracy: 0.9577 - val_loss: 0.0981
Epoch 19/40
10/10 ————— 10s 700ms/step - accuracy: 0.9937 - loss: 0.0260 - val_accuracy: 0.9577 - val_loss: 0.0963
Epoch 20/40
10/10 ————— 7s 727ms/step - accuracy: 0.9937 - loss: 0.0368 - val_accuracy: 0.9577 - val_loss: 0.0954
Epoch 21/40

```

```

10/10 ----- 6s 629ms/step - accuracy: 0.9937 - loss: 0.0314 - val_accuracy: 0.9577 - val_loss: 0.0970
Epoch 22/40
10/10 ----- 10s 638ms/step - accuracy: 0.9906 - loss: 0.0387 - val_accuracy: 0.9577 - val_loss: 0.0994
Epoch 23/40

```

```

# =====
# FIX 2 - Train CyberNetX on the CORRECTED clean data, across
# 3 fixed seeds, with full determinism. This uses your EXACT
# architecture and hyperparameters from "Image Preprocessing.ipynb"
# - nothing about the model or training config has changed, only:
#   (a) the data now comes from the collision-fixed folder (Fix 1)
#   (b) randomness is now fully pinned
#   (c) it loops over 3 seeds and records every result
#
# Run FIX 1 first. Then run this cell.
# =====
import os, random
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, Model, Input
from tensorflow.keras.applications import ResNet50, InceptionResNetV2, Xception
from tensorflow.keras.applications.resnet50 import preprocess_input as pre_res
from tensorflow.keras.applications.inception_resnet_v2 import preprocess_input as pre_inc
from tensorflow.keras.applications.xception import preprocess_input as pre_xcep
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

def set_full_determinism(seed: int):
    os.environ["PYTHONHASHSEED"] = str(seed)
    os.environ["TF_DETERMINISTIC_OPS"] = "1"
    os.environ["TF_CUDNN_DETERMINISTIC"] = "1"
    random.seed(seed); np.random.seed(seed); tf.random.set_seed(seed)
    try:
        tf.config.experimental.enable_op_determinism()
    except Exception as e:
        print("Warning: enable_op_determinism() unavailable:", e)
        print(f"[reproducibility] All seeds pinned to {seed}.")

clean = "/content/drive/MyDrive/cyberbullying_dataset_clean/"
IMG, BATCH = (224, 224), 32
SEEDS = [42, 123, 2024]

def load(split, shuffle, seed):
    return tf.keras.utils.image_dataset_from_directory(
        clean + split, image_size=IMG, batch_size=BATCH,
        label_mode="int", class_names=["bully", "no_bully"],
        shuffle=shuffle, seed=seed if shuffle else None)

def build_model():
    aug = tf.keras.Sequential([layers.RandomFlip("horizontal"),
                               layers.RandomRotation(0.1),
                               layers.RandomZoom(0.1)])

    def bb(net, name):
        m = net(include_top=False, pooling="avg", weights="imagenet")
        m.trainable = False
        m._name = name
        return m

    res, inc, xcp = bb(ResNet50, "resnet50"), bb(InceptionResNetV2, "incresv2"), bb(Xception, "xception")
    inp = Input((224, 224, 3)); a = aug(inp)
    fr = res(layers.Lambda(pre_res)(a))
    fi = inc(layers.Lambda(pre_inc)(a))
    fx = xcp(layers.Lambda(pre_xcep)(a))
    z = layers.Concatenate()([fr, fi, fx])
    z = layers.BatchNormalization()(z)
    z = layers.Dense(512, activation="relu")(z)
    z = layers.Dropout(0.5)(z)
    z = layers.Dense(256, activation="relu")(z)
    z = layers.Dropout(0.5)(z)
    out = layers.Dense(1, activation="sigmoid")(z)
    model = Model(inp, out)
    model.compile(optimizer=tf.keras.optimizers.Adam(1e-4),
                  loss="binary_crossentropy", metrics=["accuracy"])
    return model

def evaluate(model, ds):
    y_true, y_prob = [], []
    for xb, yb in ds:
        y_prob.extend(model.predict(xb, verbose=0).ravel())
        y_true.extend(yb.numpy())
    y_true, y_prob = np.array(y_true), np.array(y_prob)
    y_pred = (y_prob >= 0.5).astype(int)
    acc = round(accuracy_score(y_true, y_pred) * 100, 2)
    cm = confusion_matrix(y_true, y_pred)
    report = classification_report(y_true, y_pred, target_names=["bully", "no_bully"],
                                   digits=3, output_dict=True)

    return acc, cm, report

all_results = []
trained_models = [] # keep these around if you want to reuse one for Grad-CAM afterward

```

```

for seed in SEEDS:
    print(f"\n{' '*20} SEED {seed} {' '*20}")
    set_full_determinism(seed)

    train_ds = load("train", True, seed).prefetch(tf.data.AUTOTUNE)
    val_ds   = load("val", False, seed).prefetch(tf.data.AUTOTUNE)
    test_ds  = load("test", False, seed).prefetch(tf.data.AUTOTUNE)

    model = build_model()
    cb = [tf.keras.callbacks.EarlyStopping(patience=8, restore_best_weights=True, monitor="val_loss")]
    model.fit(train_ds, validation_data=val_ds, epochs=40, callbacks=cb, verbose=1)

    print("\n----- HONEST in-domain evaluation on the untouched TEST set -----")
    in_acc, in_cm, in_report = evaluate(model, test_ds)
    print(f"In-domain test accuracy: {in_acc}%")
    print("Confusion matrix [rows = true bully, no_bully]:\n", in_cm)

    all_results.append({"seed": seed, "in_domain_acc": in_acc,
                       "in_domain_cm": in_cm.tolist(), "in_domain_report": in_report})
    trained_models.append(model)

# -----
# PASTE YOUR EXISTING HATEFUL MEMES LOADING + EVALUATION CODE
# HERE, unchanged (the part of your "CROSS-DATASET COLLAPSE
# TEST" cell that comes after training and produces "Real
# Hateful Memes test samples: 1500" and the confusion matrix).
# It already works correctly and doesn't need any fix - just
# call it here using this loop's `model`, and append its
# accuracy/confusion matrix to all_results[-1] the same way
# as the in-domain block above, e.g.:
#
# cross_acc, cross_cm, cross_report = your_existing_eval_function(model)
# all_results[-1]["cross_acc"] = cross_acc
# all_results[-1]["cross_cm"] = cross_cm
# -----

print("\n\n===== FINAL SUMMARY =====")
in_accs = [r["in_domain_acc"] for r in all_results]
print(f"In-domain accuracy across seeds: {in_accs}")
print(f" mean = {np.mean(in_accs):.2f}%   std = {np.std(in_accs):.2f}%")
print("\nFull results (copy this whole block back to Claude):")
import json
print(json.dumps(all_results, indent=2, default=str))

```

```

===== SEED 42 =====
[reproducibility] All seeds pinned to 42.
Found 320 files belonging to 2 classes.
Found 71 files belonging to 2 classes.
Found 70 files belonging to 2 classes.
Epoch 1/40
10/10 ----- 49s 2s/step - accuracy: 0.7094 - loss: 0.6445 - val_accuracy: 0.9296 - val_loss: 0.3344
Epoch 2/40
10/10 ----- 7s 717ms/step - accuracy: 0.8625 - loss: 0.3219 - val_accuracy: 0.9437 - val_loss: 0.2297
Epoch 3/40
10/10 ----- 9s 886ms/step - accuracy: 0.9062 - loss: 0.2264 - val_accuracy: 0.9437 - val_loss: 0.1848
Epoch 4/40
10/10 ----- 9s 735ms/step - accuracy: 0.9219 - loss: 0.1895 - val_accuracy: 0.9437 - val_loss: 0.1620
Epoch 5/40
10/10 ----- 7s 737ms/step - accuracy: 0.9500 - loss: 0.1502 - val_accuracy: 0.9296 - val_loss: 0.1588
Epoch 6/40
10/10 ----- 7s 718ms/step - accuracy: 0.9438 - loss: 0.1185 - val_accuracy: 0.9437 - val_loss: 0.1471
Epoch 7/40
10/10 ----- 10s 693ms/step - accuracy: 0.9438 - loss: 0.1201 - val_accuracy: 0.9718 - val_loss: 0.1306
Epoch 8/40
10/10 ----- 7s 709ms/step - accuracy: 0.9563 - loss: 0.1136 - val_accuracy: 0.9718 - val_loss: 0.1207
Epoch 9/40
10/10 ----- 7s 698ms/step - accuracy: 0.9656 - loss: 0.0742 - val_accuracy: 0.9718 - val_loss: 0.1190
Epoch 10/40
10/10 ----- 10s 704ms/step - accuracy: 0.9906 - loss: 0.0594 - val_accuracy: 0.9577 - val_loss: 0.1157
Epoch 11/40
10/10 ----- 6s 628ms/step - accuracy: 0.9750 - loss: 0.0757 - val_accuracy: 0.9577 - val_loss: 0.1185
Epoch 12/40
10/10 ----- 10s 628ms/step - accuracy: 0.9625 - loss: 0.0854 - val_accuracy: 0.9437 - val_loss: 0.1197
Epoch 13/40
10/10 ----- 6s 652ms/step - accuracy: 0.9719 - loss: 0.0639 - val_accuracy: 0.9437 - val_loss: 0.1159
Epoch 14/40
10/10 ----- 7s 718ms/step - accuracy: 0.9812 - loss: 0.0730 - val_accuracy: 0.9437 - val_loss: 0.1070
Epoch 15/40
10/10 ----- 7s 734ms/step - accuracy: 0.9781 - loss: 0.0493 - val_accuracy: 0.9437 - val_loss: 0.1041
Epoch 16/40
10/10 ----- 7s 705ms/step - accuracy: 0.9781 - loss: 0.0494 - val_accuracy: 0.9577 - val_loss: 0.1014
Epoch 17/40
10/10 ----- 10s 715ms/step - accuracy: 0.9781 - loss: 0.0487 - val_accuracy: 0.9577 - val_loss: 0.0998
Epoch 18/40
10/10 ----- 7s 711ms/step - accuracy: 0.9875 - loss: 0.0387 - val_accuracy: 0.9577 - val_loss: 0.0981
Epoch 19/40
10/10 ----- 7s 717ms/step - accuracy: 0.9937 - loss: 0.0260 - val_accuracy: 0.9577 - val_loss: 0.0963
Epoch 20/40
10/10 ----- 7s 714ms/step - accuracy: 0.9937 - loss: 0.0368 - val_accuracy: 0.9577 - val_loss: 0.0954
Epoch 21/40
10/10 ----- 9s 620ms/step - accuracy: 0.9937 - loss: 0.0314 - val_accuracy: 0.9577 - val_loss: 0.0970
Epoch 22/40
10/10 ----- 6s 644ms/step - accuracy: 0.9906 - loss: 0.0387 - val_accuracy: 0.9577 - val_loss: 0.0994
Epoch 23/40
10/10 ----- 10s 627ms/step - accuracy: 0.9906 - loss: 0.0295 - val_accuracy: 0.9437 - val_loss: 0.1060

```

```
Epoch 24/40
10/10 ----- 7s 662ms/step - accuracy: 0.9937 - loss: 0.0247 - val_accuracy: 0.9437 - val_loss: 0.1127
Epoch 25/40
10/10 ----- 6s 634ms/step - accuracy: 0.9969 - loss: 0.0150 - val_accuracy: 0.9437 - val_loss: 0.1156
Epoch 26/40
10/10 ----- 6s 640ms/step - accuracy: 0.9986 - loss: 0.0242 - val_accuracy: 0.9296 - val_loss: 0.1104
```

```
# =====
# COMPLETE PIPELINE – run this as ONE cell in Colab.
# Does everything: force-remounts Drive, verifies the clean
# dataset is really fixed (stops immediately if not), trains
# CyberNetX across 3 seeds with full determinism, evaluates
# in-domain AND on the 1,500 real Hateful Memes images, prints
# one final summary block.
# =====

# ----- 0. Force-remount Drive (clears any stale cache) -----
from google.colab import drive
drive.mount('/content/drive', force_remount=True)

import os, json, random, subprocess, sys
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, Model, Input
from tensorflow.keras.applications import ResNet50, InceptionResNetV2, Xception
from tensorflow.keras.applications.resnet50 import preprocess_input as pre_res
from tensorflow.keras.applications.inception_resnet_v2 import preprocess_input as pre_inc
from tensorflow.keras.applications.xception import preprocess_input as pre_xcep
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

# ----- 1. HARD CHECK: stop now if the clean data isn't really fixed -----
clean = "/content/drive/MyDrive/cyberbullying_dataset_clean/"
counts = {}
for split in ["train", "val", "test"]:
    n = 0
    for cls in ["bully", "no_bully"]:
        d = os.path.join(clean, split, cls)
        n += len(os.listdir(d)) if os.path.isdir(d) else 0
    counts[split] = n
    print(f"{split}: {n} files on disk")

expected = {"train": 330, "val": 71, "test": 71}
if counts != expected:
    raise RuntimeError(
        f"STOP: clean dataset counts are {counts}, expected {expected}. "
        "Drive is still showing stale/incorrect data – do not proceed with "
        "training until this matches exactly. Try Runtime > Restart runtime, "
        "then run this cell again from the top."
    )
print("Data check passed – 330/71/71 confirmed. Proceeding.\n")

# ----- 2. Get Hateful Memes (skips download if already present) -----
hm_root = "/content/hateful_memes/hateful_memes"
if not os.path.exists(hm_root):
    print("Downloading Hateful Memes (~4GB, ~2 min)...")
    os.makedirs("/root/.kaggle", exist_ok=True)
    drive_key = "/content/drive/MyDrive/kaggle.json"
    if os.path.exists(drive_key):
        os.system(f"cp '{drive_key}' /root/.kaggle/kaggle.json")
    os.system("chmod 600 /root/.kaggle/kaggle.json")
    subprocess.run([sys.executable, "-m", "pip", "install", "-q", "kaggle"])
    os.system("kaggle datasets download -d williamberrios/hateful-memes -p /content")
    os.system("unzip -q /content/hateful-memes.zip -d /content/hateful_memes")
    print("Hateful Memes ready:", os.path.exists(hm_root))

def load_hateful_memes():
    """Loads dev_seen + test_seen (1,500 images), matching the manuscript's protocol.
    JSONL label: 1 = hateful. Mapped to model convention: 0 = bully/harmful,
    1 = no_bully/not-harmful (to match class_names=['bully', 'no_bully'] used
    for the stock data), i.e. model_label = 1 - jsonl_label."""
    records = []
    for fname in ["dev_seen.jsonl", "test_seen.jsonl"]:
        with open(os.path.join(hm_root, fname)) as f:
            for line in f:
                rec = json.loads(line)
                records.append(rec)
    print(f"Loaded {len(records)} Hateful Memes records (dev_seen + test_seen).")

    images, labels = [], []
    for rec in records:
        img_path = os.path.join(hm_root, rec["img"])
        try:
            img = tf.keras.utils.load_img(img_path, target_size=(224, 224))
            arr = tf.keras.utils.img_to_array(img)
            images.append(arr)
            labels.append(1 - int(rec["label"])) # map to bully=0 / no_bully=1
        except Exception:
            pass
    X = np.array(images)
    y = np.array(labels)
    print(f"Prepared {len(X)} images. not-harmful(no_bully)={int((y==1).sum())}, "
          f"harmful(bully)={int((y==0).sum())}")
```

```

    return X, y

hm_X, hm_y = load_hateful_memes()

# ----- 3. Determinism -----
def set_full_determinism(seed: int):
    os.environ["PYTHONHASHSEED"] = str(seed)
    os.environ["TF_DETERMINISTIC_OPS"] = "1"
    os.environ["TF_CUDNN_DETERMINISTIC"] = "1"
    random.seed(seed); np.random.seed(seed); tf.random.set_seed(seed)
    try:
        tf.config.experimental.enable_op_determinism()
    except Exception as e:
        print("Warning: enable_op_determinism() unavailable:", e)
        print(f"[reproducibility] All seeds pinned to {seed}.")

# ----- 4. Model (exact architecture from the manuscript) -----
IMG, BATCH = (224, 224), 32

def load_split(split, shuffle, seed):
    return tf.keras.utils.image_dataset_from_directory(
        clean + split, image_size=IMG, batch_size=BATCH,
        label_mode="int", class_names=["bully", "no_bully"],
        shuffle=shuffle, seed=seed if shuffle else None)

def build_model():
    aug = tf.keras.Sequential([layers.RandomFlip("horizontal"),
                               layers.RandomRotation(0.1),
                               layers.RandomZoom(0.1)])

    def bb(net, name):
        m = net(include_top=False, pooling="avg", weights="imagenet")
        m.trainable = False; m._name = name
        return m

    res, inc, xcp = bb(ResNet50, "resnet50"), bb(InceptionResNetV2, "incresv2"), bb(Xception, "xception")
    inp = Input((224, 224, 3)); a = aug(inp)
    fr = res(layers.Lambda(pre_res)(a))
    fi = inc(layers.Lambda(pre_inc)(a))
    fx = xcp(layers.Lambda(pre_xcep)(a))
    z = layers.Concatenate()([fr, fi, fx])
    z = layers.BatchNormalization()(z)
    z = layers.Dense(512, activation="relu")(z)
    z = layers.Dropout(0.5)(z)
    z = layers.Dense(256, activation="relu")(z)
    z = layers.Dropout(0.5)(z)
    out = layers.Dense(1, activation="sigmoid")(z)
    model = Model(inp, out)
    model.compile(optimizer=tf.keras.optimizers.Adam(1e-4),
                  loss="binary_crossentropy", metrics=["accuracy"])

    return model

def evaluate_ds(model, ds):
    y_true, y_prob = [], []
    for xb, yb in ds:
        y_prob.extend(model.predict(xb, verbose=0).ravel())
        y_true.extend(yb.numpy())
    y_true, y_prob = np.array(y_true), np.array(y_prob)
    y_pred = (y_prob >= 0.5).astype(int)
    acc = round(accuracy_score(y_true, y_pred) * 100, 2)
    cm = confusion_matrix(y_true, y_pred)
    report = classification_report(y_true, y_pred, target_names=["bully", "no_bully"],
                                  digits=3, output_dict=True)

    return acc, cm, report

def evaluate_array(model, X, y):
    y_prob = model.predict(X, batch_size=32, verbose=0).ravel()
    y_pred = (y_prob >= 0.5).astype(int)
    acc = round(accuracy_score(y, y_pred) * 100, 2)
    cm = confusion_matrix(y, y_pred)
    report = classification_report(y, y_pred, target_names=["bully", "no_bully"],
                                  digits=3, output_dict=True)

    return acc, cm, report

# ----- 5. Multi-seed loop -----
SEEDS = [42, 123, 2024]
all_results = []

for seed in SEEDS:
    print(f"\n{' '*20} SEED {seed} {' '*20}")
    set_full_determinism(seed)

    train_ds = load_split("train", True, seed).prefetch(tf.data.AUTOTUNE)
    val_ds = load_split("val", False, seed).prefetch(tf.data.AUTOTUNE)
    test_ds = load_split("test", False, seed).prefetch(tf.data.AUTOTUNE)

    model = build_model()
    cb = [tf.keras.callbacks.EarlyStopping(patience=8, restore_best_weights=True, monitor="val_loss")]
    model.fit(train_ds, validation_data=val_ds, epochs=40, callbacks=cb, verbose=1)

    print("\n----- In-domain (clean stock test set) -----")
    in_acc, in_cm, in_report = evaluate_ds(model, test_ds)
    print(f"In-domain accuracy: {in_acc}% | confusion matrix:\n{in_cm}")

```

```

print("\n----- Cross-dataset (1,500 real Hateful Memes) -----")
cross_acc, cross_cm, cross_report = evaluate_array(model, hm_X, hm_y)
print(f"Cross-dataset accuracy: {cross_acc}% | confusion matrix:\n{cross_cm}")

all_results.append({
    "seed": seed,
    "in_domain_acc": in_acc, "in_domain_cm": in_cm.tolist(), "in_domain_report": in_report,
    "cross_acc": cross_acc, "cross_cm": cross_cm.tolist(), "cross_report": cross_report,
})

# ----- 6. Final summary -----
in_accs = [r["in_domain_acc"] for r in all_results]
cross_accs = [r["cross_acc"] for r in all_results]
print("\n\n===== FINAL SUMMARY (send this whole block back) =====")
print(f"In-domain: {in_accs} mean={np.mean(in_accs):.2f}% std={np.std(in_accs):.2f}%")
print(f"Cross-dataset: {cross_accs} mean={np.mean(cross_accs):.2f}% std={np.std(cross_accs):.2f}%")
print("\nFull JSON:")
print(json.dumps(all_results, indent=2, default=str))

```

Mounted at /content/drive
train: 330 files on disk
val: 71 files on disk
test: 71 files on disk
Data check passed – 330/71/71 confirmed. Proceeding.

Downloading Hateful Memes (~4GB, ~2 min)...
Hateful Memes ready: True
Loaded 1500 Hateful Memes records (dev_seen + test_seen).
Prepared 1500 images. not-harmful(no_bully)=763, harmful(bully)=737

```

===== SEED 42 =====
[reproducibility] All seeds pinned to 42.
Found 320 files belonging to 2 classes.
Found 71 files belonging to 2 classes.
Found 70 files belonging to 2 classes.
Epoch 1/40
10/10 ----- 48s 2s/step - accuracy: 0.7094 - loss: 0.6445 - val_accuracy: 0.9296 - val_loss: 0.3344
Epoch 2/40
10/10 ----- 7s 729ms/step - accuracy: 0.8625 - loss: 0.3219 - val_accuracy: 0.9437 - val_loss: 0.2297
Epoch 3/40
10/10 ----- 7s 752ms/step - accuracy: 0.9062 - loss: 0.2264 - val_accuracy: 0.9437 - val_loss: 0.1848
Epoch 4/40
10/10 ----- 7s 747ms/step - accuracy: 0.9219 - loss: 0.1895 - val_accuracy: 0.9437 - val_loss: 0.1620
Epoch 5/40
10/10 ----- 7s 748ms/step - accuracy: 0.9500 - loss: 0.1502 - val_accuracy: 0.9296 - val_loss: 0.1588
Epoch 6/40
10/10 ----- 7s 715ms/step - accuracy: 0.9438 - loss: 0.1185 - val_accuracy: 0.9437 - val_loss: 0.1471
Epoch 7/40
10/10 ----- 7s 726ms/step - accuracy: 0.9438 - loss: 0.1201 - val_accuracy: 0.9718 - val_loss: 0.1306
Epoch 8/40
10/10 ----- 7s 715ms/step - accuracy: 0.9563 - loss: 0.1136 - val_accuracy: 0.9718 - val_loss: 0.1207
Epoch 9/40
10/10 ----- 7s 726ms/step - accuracy: 0.9656 - loss: 0.0742 - val_accuracy: 0.9718 - val_loss: 0.1190
Epoch 10/40
10/10 ----- 7s 703ms/step - accuracy: 0.9906 - loss: 0.0594 - val_accuracy: 0.9577 - val_loss: 0.1157
Epoch 11/40
10/10 ----- 6s 634ms/step - accuracy: 0.9750 - loss: 0.0757 - val_accuracy: 0.9577 - val_loss: 0.1185
Epoch 12/40
10/10 ----- 6s 629ms/step - accuracy: 0.9625 - loss: 0.0854 - val_accuracy: 0.9437 - val_loss: 0.1197
Epoch 13/40
10/10 ----- 10s 640ms/step - accuracy: 0.9719 - loss: 0.0639 - val_accuracy: 0.9437 - val_loss: 0.1159
Epoch 14/40
10/10 ----- 7s 712ms/step - accuracy: 0.9812 - loss: 0.0730 - val_accuracy: 0.9437 - val_loss: 0.1070
Epoch 15/40
10/10 ----- 7s 739ms/step - accuracy: 0.9781 - loss: 0.0493 - val_accuracy: 0.9437 - val_loss: 0.1041
Epoch 16/40
10/10 ----- 7s 712ms/step - accuracy: 0.9781 - loss: 0.0494 - val_accuracy: 0.9577 - val_loss: 0.1014
Epoch 17/40
10/10 ----- 7s 729ms/step - accuracy: 0.9781 - loss: 0.0487 - val_accuracy: 0.9577 - val_loss: 0.0998
Epoch 18/40
10/10 ----- 10s 720ms/step - accuracy: 0.9875 - loss: 0.0387 - val_accuracy: 0.9577 - val_loss: 0.0981
Epoch 19/40
10/10 ----- 7s 725ms/step - accuracy: 0.9937 - loss: 0.0260 - val_accuracy: 0.9577 - val_loss: 0.0963
Epoch 20/40
10/10 ----- 7s 716ms/step - accuracy: 0.9937 - loss: 0.0368 - val_accuracy: 0.9577 - val_loss: 0.0954
Epoch 21/40
10/10 ----- 6s 645ms/step - accuracy: 0.9937 - loss: 0.0314 - val_accuracy: 0.9577 - val_loss: 0.0970

```

```

# =====
# FIX 3 – Find and repair the 11 "invisible" files
# Keras's image_dataset_from_directory only recognizes these
# extensions: .bmp .gif .jpeg .jpg .png (case-insensitive).
# Your files passed PIL during the DINOv2 audit (which is more
# permissive) and got copied into the clean folder, but some of
# them likely have an extension Keras's loader doesn't accept
# (e.g. .jif, .webp, or no extension) – that's almost certainly
# your missing 10 train + 1 test file.
#
# This script finds them, then REPAIRS them by re-saving as a
# proper .jpg (Keras will then see them).
# =====
import pathlib
from PIL import Image

clean = pathlib.Path("/content/drive/MyDrive/cyberbullying_dataset_clean/")

```

```

ALLOWED = {".bmp", ".gif", ".jpeg", ".jpg", ".png"}

print("Scanning for files Keras's loader would skip...\n")
suspects = []
for split in ["train", "val", "test"]:
    for cls in ["bully", "no_bully"]:
        d = clean / split / cls
        if not d.is_dir():
            continue
        for f in d.iterdir():
            if f.suffix.lower() not in ALLOWED:
                suspects.append(f)
                print(f"SUSPECT: {f} (extension: '{f.suffix}')"")

print(f"\nFound {len(suspects)} suspect file(s).")

if len(suspects) == 0:
    print("\nNo extension problems found. The mismatch must have a different "
          "cause -- run the deeper diagnostic below instead.")
else:
    print("\nRepairing: re-saving each as a proper .jpg file...")
    for f in suspects:
        try:
            img = Image.open(f).convert("RGB")
            new_path = f.with_suffix(".jpg")
            img.save(new_path, "JPEG")
            if new_path != f:
                f.unlink() # remove the old, unrecognized file
                print(f"fixed: {f.name} -> {new_path.name}")
        except Exception as e:
            print(f"COULD NOT FIX {f}: {e}")

    print("\nRe-verifying counts after repair:")
    for split in ["train", "val", "test"]:
        total = 0
        for cls in ["bully", "no_bully"]:
            d = clean / split / cls
            n = len(list(d.iterdir())) if d.is_dir() else 0
            total += n
        print(f" {split}: {total} files")

# -----
# DEEPER DIAGNOSTIC (only needed if the scan above found 0
# suspects, meaning the extensions look fine but Keras still
# skips them -- likely a truncated/corrupted image instead):
#
# import tensorflow as tf
# for split in ["train", "test"]:
#     for cls in ["bully", "no_bully"]:
#         d = clean / split / cls
#         for f in d.iterdir():
#             try:
#                 raw = tf.io.read_file(str(f))
#                 tf.io.decode_image(raw)
#             except Exception as e:
#                 print(f"UNREADABLE BY TF: {f} ({e})")
# -----

```

Scanning for files Keras's loader would skip...

```

SUSPECT: /content/drive/MyDrive/cyberbullying_dataset_clean/train/bully/00000_cyber-bullying-student-girls-two-making-smartphone-to-girl-re
SUSPECT: /content/drive/MyDrive/cyberbullying_dataset_clean/train/bully/00211_image-20150415-31666-139ffjn.avif (extension: '.avif')
SUSPECT: /content/drive/MyDrive/cyberbullying_dataset_clean/train/bully/00006_Blog-1_Hero.webp (extension: '.webp')
SUSPECT: /content/drive/MyDrive/cyberbullying_dataset_clean/train/bully/00481_teenage-girl-being-bullied-by-260nw-523091407.webp (extensi
SUSPECT: /content/drive/MyDrive/cyberbullying_dataset_clean/train/no_bully/00225_70080_shutterstock_479264641.avif (extension: '.avif')
SUSPECT: /content/drive/MyDrive/cyberbullying_dataset_clean/train/no_bully/00495_racist-bullying.webp (extension: '.webp')
SUSPECT: /content/drive/MyDrive/cyberbullying_dataset_clean/train/no_bully/00228_generic-employment-hero.avif (extension: '.avif')
SUSPECT: /content/drive/MyDrive/cyberbullying_dataset_clean/train/no_bully/00222_Cyberbullying.webp (extension: '.webp')
SUSPECT: /content/drive/MyDrive/cyberbullying_dataset_clean/train/no_bully/00498_shutterstock_1443616970-scaled.webp (extension: '.webp')
SUSPECT: /content/drive/MyDrive/cyberbullying_dataset_clean/train/no_bully/00226_7eff9f_c2f2a8fb1fd84e41997209746fccee00_mv2.avif (extens
SUSPECT: /content/drive/MyDrive/cyberbullying_dataset_clean/test/no_bully/00497_teenstechfeatured.webp (extension: '.webp')

```

Found 11 suspect file(s).

Repairing: re-saving each as a proper .jpg file...

```

fixed: 00000_cyber-bullying-student-girls-two-making-smartphone-to-girl-reacting-angry-40834232.webp -> 00000_cyber-bullying-student-girl
fixed: 00211_image-20150415-31666-139ffjn.avif -> 00211_image-20150415-31666-139ffjn.jpg
fixed: 00006_Blog-1_Hero.webp -> 00006_Blog-1_Hero.jpg
fixed: 00481_teenage-girl-being-bullied-by-260nw-523091407.webp -> 00481_teenage-girl-being-bullied-by-260nw-523091407.jpg
fixed: 00225_70080_shutterstock_479264641.avif -> 00225_70080_shutterstock_479264641.jpg
fixed: 00495_racist-bullying.webp -> 00495_racist-bullying.jpg
fixed: 00228_generic-employment-hero.avif -> 00228_generic-employment-hero.jpg
fixed: 00222_Cyberbullying.webp -> 00222_Cyberbullying.jpg
fixed: 00498_shutterstock_1443616970-scaled.webp -> 00498_shutterstock_1443616970-scaled.jpg
fixed: 00226_7eff9f_c2f2a8fb1fd84e41997209746fccee00_mv2.avif -> 00226_7eff9f_c2f2a8fb1fd84e41997209746fccee00_mv2.jpg
fixed: 00497_teenstechfeatured.webp -> 00497_teenstechfeatured.jpg

```

Re-verifying counts after repair:
train: 330 files
val: 71 files
test: 71 files

```

# =====
# COMPLETE PIPELINE – run this as ONE cell in Colab.
# Does everything: force-remounts Drive, verifies the clean
# dataset is really fixed (stops immediately if not), trains
# CyberNetX across 3 seeds with full determinism, evaluates
# in-domain AND on the 1,500 real Hateful Memes images, prints
# one final summary block.
# =====

# ----- 0. Force-remount Drive (clears any stale cache) -----
from google.colab import drive
drive.mount('/content/drive', force_remount=True)

import os, json, random, subprocess, sys
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, Model, Input
from tensorflow.keras.applications import ResNet50, InceptionResNetV2, Xception
from tensorflow.keras.applications.resnet50 import preprocess_input as pre_res
from tensorflow.keras.applications.inception_resnet_v2 import preprocess_input as pre_inc
from tensorflow.keras.applications.xception import preprocess_input as pre_xcep
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

# ----- 1. HARD CHECK: stop now if the clean data isn't really fixed -----
clean = "/content/drive/MyDrive/cyberbullying_dataset_clean/"
counts = {}
for split in ["train", "val", "test"]:
    n = 0
    for cls in ["bully", "no_bully"]:
        d = os.path.join(clean, split, cls)
        n += len(os.listdir(d)) if os.path.isdir(d) else 0
    counts[split] = n
    print(f"{split}: {n} files on disk")

expected = {"train": 330, "val": 71, "test": 71}
if counts != expected:
    raise RuntimeError(
        f"STOP: clean dataset counts are {counts}, expected {expected}. "
        "Drive is still showing stale/incorrect data – do not proceed with "
        "training until this matches exactly. Try Runtime > Restart runtime, "
        "then run this cell again from the top."
    )
print("Data check passed – 330/71/71 confirmed. Proceeding.\n")

# ----- 2. Get Hateful Memes (skips download if already present) -----
hm_root = "/content/hateful_memes/hateful_memes"
if not os.path.exists(hm_root):
    print("Downloading Hateful Memes (~4GB, ~2 min)...")
    os.makedirs("/root/.kaggle", exist_ok=True)
    drive_key = "/content/drive/MyDrive/kaggle.json"
    if os.path.exists(drive_key):
        os.system(f"cp '{drive_key}' /root/.kaggle/kaggle.json")
    os.system("chmod 600 /root/.kaggle/kaggle.json")
    subprocess.run([sys.executable, "-m", "pip", "install", "-q", "kaggle"])
    os.system("kaggle datasets download -d williamberrios/hateful-memes -p /content")
    os.system("unzip -q /content/hateful-memes.zip -d /content/hateful_memes")
    print("Hateful Memes ready:", os.path.exists(hm_root))

def load_hateful_memes():
    """Loads dev_seen + test_seen (1,500 images), matching the manuscript's protocol.
    JSONL label: 1 = hateful. Mapped to model convention: 0 = bully/harmful,
    1 = no_bully/not-harmful (to match class_names=['bully', 'no_bully'] used
    for the stock data), i.e. model_label = 1 - jsonl_label."""
    records = []
    for fname in ["dev_seen.jsonl", "test_seen.jsonl"]:
        with open(os.path.join(hm_root, fname)) as f:
            for line in f:
                rec = json.loads(line)
                records.append(rec)
    print(f"Loaded {len(records)} Hateful Memes records (dev_seen + test_seen).")

    images, labels = [], []
    for rec in records:
        img_path = os.path.join(hm_root, rec["img"])
        try:
            img = tf.keras.utils.load_img(img_path, target_size=(224, 224))
            arr = tf.keras.utils.img_to_array(img)
            images.append(arr)
            labels.append(1 - int(rec["label"])) # map to bully=0 / no_bully=1
        except Exception:
            pass
    X = np.array(images)
    y = np.array(labels)
    print(f"Prepared {len(X)} images. not-harmful(no_bully)={int((y==1).sum())}, "
          f"harmful(bully)={int((y==0).sum())}")
    return X, y

hm_X, hm_y = load_hateful_memes()

# ----- 3. Determinism -----
def set_full_determinism(seed: int):

```

```

os.environ["PYTHONHASHSEED"] = str(seed)
os.environ["TF_DETERMINISTIC_OPS"] = "1"
os.environ["TF_CUDNN_DETERMINISTIC"] = "1"
random.seed(seed); np.random.seed(seed); tf.random.set_seed(seed)
try:
    tf.config.experimental.enable_op_determinism()
except Exception as e:
    print("Warning: enable_op_determinism() unavailable:", e)
print(f"[reproducibility] All seeds pinned to {seed}.")

# ----- 4. Model (exact architecture from the manuscript) -----
IMG, BATCH = (224, 224), 32

def load_split(split, shuffle, seed):
    return tf.keras.utils.image_dataset_from_directory(
        clean + split, image_size=IMG, batch_size=BATCH,
        label_mode="int", class_names=["bully", "no_bully"],
        shuffle=shuffle, seed=seed if shuffle else None)

def build_model():
    aug = tf.keras.Sequential([layers.RandomFlip("horizontal"),
                               layers.RandomRotation(0.1),
                               layers.RandomZoom(0.1)])

    def bb(net, name):
        m = net(include_top=False, pooling="avg", weights="imagenet")
        m.trainable = False; m._name = name
        return m

    res, inc, xcp = bb(ResNet50, "resnet50"), bb(InceptionResNetV2, "incresv2"), bb(Xception, "xception")
    inp = Input((224, 224, 3)); a = aug(inp)
    fr = res(layers.Lambda(pre_res))(a)
    fi = inc(layers.Lambda(pre_inc))(a)
    fx = xcp(layers.Lambda(pre_xcp))(a)
    z = layers.Concatenate()([fr, fi, fx])
    z = layers.BatchNormalization()(z)
    z = layers.Dense(512, activation="relu")(z)
    z = layers.Dropout(0.5)(z)
    z = layers.Dense(256, activation="relu")(z)
    z = layers.Dropout(0.5)(z)
    out = layers.Dense(1, activation="sigmoid")(z)
    model = Model(inp, out)
    model.compile(optimizer=tf.keras.optimizers.Adam(1e-4),
                  loss="binary_crossentropy", metrics=["accuracy"])

    return model

def evaluate_ds(model, ds):
    y_true, y_prob = [], []
    for xb, yb in ds:
        y_prob.extend(model.predict(xb, verbose=0).ravel())
        y_true.extend(yb.numpy())
    y_true, y_prob = np.array(y_true), np.array(y_prob)
    y_pred = (y_prob >= 0.5).astype(int)
    acc = round(accuracy_score(y_true, y_pred) * 100, 2)
    cm = confusion_matrix(y_true, y_pred)
    report = classification_report(y_true, y_pred, target_names=["bully", "no_bully"],
                                  digits=3, output_dict=True)

    return acc, cm, report

def evaluate_array(model, X, y):
    y_prob = model.predict(X, batch_size=32, verbose=0).ravel()
    y_pred = (y_prob >= 0.5).astype(int)
    acc = round(accuracy_score(y, y_pred) * 100, 2)
    cm = confusion_matrix(y, y_pred)
    report = classification_report(y, y_pred, target_names=["bully", "no_bully"],
                                  digits=3, output_dict=True)

    return acc, cm, report

# ----- 5. Multi-seed loop -----
SEEDS = [42, 123, 2024]
all_results = []

for seed in SEEDS:
    print(f"\n{' '*20} SEED {seed} {' '*20}")
    set_full_determinism(seed)

    train_ds = load_split("train", True, seed).prefetch(tf.data.AUTOTUNE)
    val_ds = load_split("val", False, seed).prefetch(tf.data.AUTOTUNE)
    test_ds = load_split("test", False, seed).prefetch(tf.data.AUTOTUNE)

    model = build_model()
    cb = [tf.keras.callbacks.EarlyStopping(patience=8, restore_best_weights=True, monitor="val_loss")]
    model.fit(train_ds, validation_data=val_ds, epochs=40, callbacks=cb, verbose=1)

    print("\n----- In-domain (clean stock test set) -----")
    in_acc, in_cm, in_report = evaluate_ds(model, test_ds)
    print(f"In-domain accuracy: {in_acc}% | confusion matrix:\n{in_cm}")

    print("\n----- Cross-dataset (1,500 real Hateful Memes) -----")
    cross_acc, cross_cm, cross_report = evaluate_array(model, hm_X, hm_y)
    print(f"Cross-dataset accuracy: {cross_acc}% | confusion matrix:\n{cross_cm}")

    all_results.append({

```

```

"seed": seed,
"in_domain_acc": in_acc, "in_domain_cm": in_cm.tolist(), "in_domain_report": in_report,
"cross_acc": cross_acc, "cross_cm": cross_cm.tolist(), "cross_report": cross_report,
})

# ----- 6. Final summary -----
in_accs = [r["in_domain_acc"] for r in all_results]
cross_accs = [r["cross_acc"] for r in all_results]
print("\n\n===== FINAL SUMMARY (send this whole block back) =====")
print(f"In-domain:      {in_accs} mean={np.mean(in_accs):.2f}% std={np.std(in_accs):.2f}%")
print(f"Cross-dataset: {cross_accs} mean={np.mean(cross_accs):.2f}% std={np.std(cross_accs):.2f}%")
print("\nFull JSON:")
print(json.dumps(all_results, indent=2, default=str))

```

Mounted at /content/drive
train: 330 files on disk
val: 71 files on disk
test: 71 files on disk
Data check passed – 330/71/71 confirmed. Proceeding.

Hateful Memes ready: True
Loaded 1500 Hateful Memes records (dev_seen + test_seen).
Prepared 1500 images. not-harmful(no_bully)=763, harmful(bully)=737

```

===== SEED 42 =====
[reproducibility] All seeds pinned to 42.
Found 330 files belonging to 2 classes.
Found 71 files belonging to 2 classes.
Found 71 files belonging to 2 classes.
Epoch 1/40
11/11 ----- 49s 2s/step - accuracy: 0.6970 - loss: 0.6517 - val_accuracy: 0.9437 - val_loss: 0.3174
Epoch 2/40
11/11 ----- 8s 719ms/step - accuracy: 0.8636 - loss: 0.3147 - val_accuracy: 0.9577 - val_loss: 0.2224
Epoch 3/40
11/11 ----- 7s 684ms/step - accuracy: 0.8909 - loss: 0.2900 - val_accuracy: 0.9437 - val_loss: 0.2042
Epoch 4/40
11/11 ----- 8s 707ms/step - accuracy: 0.9394 - loss: 0.1589 - val_accuracy: 0.9296 - val_loss: 0.1907
Epoch 5/40
11/11 ----- 7s 678ms/step - accuracy: 0.9485 - loss: 0.1441 - val_accuracy: 0.9577 - val_loss: 0.1736
Epoch 6/40
11/11 ----- 7s 680ms/step - accuracy: 0.9242 - loss: 0.1724 - val_accuracy: 0.9437 - val_loss: 0.1680
Epoch 7/40
11/11 ----- 7s 661ms/step - accuracy: 0.9515 - loss: 0.1145 - val_accuracy: 0.9577 - val_loss: 0.1502
Epoch 8/40
11/11 ----- 7s 669ms/step - accuracy: 0.9333 - loss: 0.1591 - val_accuracy: 0.9296 - val_loss: 0.1362
Epoch 9/40
11/11 ----- 7s 663ms/step - accuracy: 0.9576 - loss: 0.0981 - val_accuracy: 0.9437 - val_loss: 0.1288
Epoch 10/40
11/11 ----- 7s 595ms/step - accuracy: 0.9485 - loss: 0.1205 - val_accuracy: 0.9155 - val_loss: 0.1366
Epoch 11/40
11/11 ----- 6s 579ms/step - accuracy: 0.9576 - loss: 0.1113 - val_accuracy: 0.9296 - val_loss: 0.1387
Epoch 12/40
11/11 ----- 7s 601ms/step - accuracy: 0.9697 - loss: 0.0651 - val_accuracy: 0.9296 - val_loss: 0.1324
Epoch 13/40
11/11 ----- 6s 587ms/step - accuracy: 0.9545 - loss: 0.1128 - val_accuracy: 0.9296 - val_loss: 0.1410
Epoch 14/40
11/11 ----- 7s 605ms/step - accuracy: 0.9818 - loss: 0.0744 - val_accuracy: 0.9296 - val_loss: 0.1373
Epoch 15/40
11/11 ----- 6s 591ms/step - accuracy: 0.9788 - loss: 0.0599 - val_accuracy: 0.9296 - val_loss: 0.1391
Epoch 16/40
11/11 ----- 7s 665ms/step - accuracy: 0.9636 - loss: 0.0948 - val_accuracy: 0.9296 - val_loss: 0.1465
Epoch 17/40
11/11 ----- 9s 584ms/step - accuracy: 0.9667 - loss: 0.0934 - val_accuracy: 0.9296 - val_loss: 0.1522

```

```

----- In-domain (clean stock test set) -----
In-domain accuracy: 95.77% | confusion matrix:
[[31  2]
 [ 1 37]]

```

```

----- Cross-dataset (1,500 real Hateful Memes) -----
Cross-dataset accuracy: 50.47% | confusion matrix:
[[233 504]

```

```

# =====
# THRESHOLD SENSITIVITY SWEEP (fills Table 1a, Section 2.2.1)
#
# Recomputes DINOv2 embeddings on the ORIGINAL 542-image dataset
# (same embedding code as your audit notebook), then sweeps the
# near-duplicate similarity threshold across several values instead
# of just 0.90, so you can show Reviewer A where the count
# stabilizes.
#
# Includes a built-in sanity check: at threshold 0.90 it should
# reproduce your known audit numbers (80 near-duplicate pairs,
# 32 leakage pairs) exactly. If it doesn't, something upstream
# has changed and the sweep isn't trustworthy yet.
# =====
import subprocess, sys
subprocess.run([sys.executable, "-m", "pip", "install", "-q", "cleanlab"])

import os, glob
import numpy as np
import torch
from PIL import Image
import torchvision.transforms as T

```

```

base = "/content/drive/MyDrive/cyberbullying_dataset/"

def gather(split):
    items = []
    sd = os.path.join(base, split)
    if not os.path.isdir(sd):
        return items
    for cls in sorted(os.listdir(sd)):
        cdir = os.path.join(sd, cls)
        if not os.path.isdir(cdir):
            continue
        for p in glob.glob(os.path.join(cdir, "*")):
            items.append({"path": p, "class_name": cls, "split": split})
    return items

data = gather("train") + gather("val")
classes = sorted(set(d["class_name"] for d in data))
c2i = {c: i for i, c in enumerate(classes)}
for d in data:
    d["label"] = c2i[d["class_name"]]

print(f"Loaded {len(data)} original images "
      f"({sum(1 for d in data if d['split']=='train')} train, "
      f"{sum(1 for d in data if d['split']=='val')} val).")

device = "cuda" if torch.cuda.is_available() else "cpu"
dino = torch.hub.load('facebookresearch/dinov2', 'dinov2_vitb14').eval().to(device)
tf = T.Compose([T.Resize((224, 224)), T.ToTensor(),
                T.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])])

embs, splits, paths, bi, bm = [], [], [], [], []
def flush():
    if not bi:
        return
    x = torch.stack(bi).to(device)
    with torch.no_grad():
        f = dino(x).cpu().numpy()
    for k, (sp, pth) in enumerate(bm):
        embs.append(f[k]); splits.append(sp); paths.append(pth)
    bi.clear(); bm.clear()

for d in data:
    try:
        im = Image.open(d["path"]).convert("RGB")
        bi.append(tf(im)); bm.append((d["split"], d["path"]))
        if len(bi) == 32:
            flush()
    except Exception:
        pass
flush()

X = np.array(embs)
splits = np.array(splits)
paths = np.array(paths)
print(f"Embedded {len(paths)} images | shape {X.shape}")

# ----- Vectorized pairwise similarity -----
Xn = X / (np.linalg.norm(X, axis=1, keepdims=True) + 1e-9)
sims = Xn @ Xn.T
n = sims.shape[0]
iu = np.triu_indices(n, k=1) # every unordered pair once
pair_sims = sims[iu]
pair_i, pair_j = iu
pair_splits_diff = splits[pair_i] != splits[pair_j]

THRESHOLDS = [0.85, 0.88, 0.90, 0.92, 0.95]

print("\nSweeping thresholds...\n")
print(f"{'Threshold':>10} | {'Near-dup pairs':>15} | {'Leakage pairs':>14}")
print("-" * 46)
results = []
for t in THRESHOLDS:
    mask = pair_sims >= t
    dup = int(mask.sum())
    leak = int(np.sum(mask & pair_splits_diff))
    results.append((t, dup, leak))
    marker = " <-- used in paper" if t == 0.90 else ""
    print(f"{t:>10.2f} | {dup:>15d} | {leak:>14d}{marker}")

# ----- Sanity check -----
used = [(d, l) for t, d, l in results if t == 0.90][0]
print("\n--- Sanity check ---")
if used == (80, 32):
    print("PASSED: threshold 0.90 reproduces the known audit numbers (80 pairs, 32 leakage).")
    print("The sweep above is trustworthy -- copy it directly into Table 1a.")
else:
    print(f"WARNING: threshold 0.90 gave {used}, expected (80, 32).")
    print("Something upstream differs from your original audit run (e.g. a different")
    print("image subset was loaded). Don't use this sweep until this is resolved --")
    print("send this exact output back to Claude before updating the paper.")

```

Loaded 542 original images (435 train, 107 val).
 Using cache found in /root/.cache/torch/hub/facebookresearch_dinov2_main
 Embedded 542 images | shape (542, 768)

Sweeping thresholds...

Threshold	Near-dup pairs	Leakage pairs
0.85	338	121
0.88	139	51
0.90	80	32 <-- used in paper
0.92	54	21
0.95	30	15

--- Sanity check ---

PASSED: threshold 0.90 reproduces the known audit numbers (80 pairs, 32 leakage).
 The sweep above is trustworthy -- copy it directly into Table 1a.

```
# =====
# PERCEPTUAL-HASH BASELINE (fills Section 2.2.2 + Table 1)
#
# Computes a conventional perceptual-hash (pHash) near-duplicate
# and leakage baseline on the same original 542-image dataset used
# for the DINOv2 audit, so the two are directly comparable.
#
# Uses imagehash's phash (DCT-based, 64-bit) with a Hamming-distance
# threshold of 5 -- a standard, commonly used cutoff for "near
# duplicate" in image-deduplication literature. A short sweep across
# a few nearby thresholds is printed too, so you can see how sensitive
# the count is and pick a defensible value if 5 doesn't feel right.
# =====
import subprocess, sys
subprocess.run([sys.executable, "-m", "pip", "install", "-q", "imagehash"])

import os, glob
import numpy as np
from PIL import Image
import imagehash

base = "/content/drive/MyDrive/cyberbullying_dataset/"

def gather(split):
    items = []
    sd = os.path.join(base, split)
    if not os.path.isdir(sd):
        return items
    for cls in sorted(os.listdir(sd)):
        cdir = os.path.join(sd, cls)
        if not os.path.isdir(cdir):
            continue
        for p in glob.glob(os.path.join(cdir, "*")):
            items.append({"path": p, "split": split})
    return items

data = gather("train") + gather("val")
print(f"Loaded {len(data)} original images "
      f"({sum(1 for d in data if d['split']=='train')} train, "
      f"{sum(1 for d in data if d['split']=='val')} val).")

# ----- Compute perceptual hashes -----
hashes, splits, paths = [], [], []
for d in data:
    try:
        im = Image.open(d["path"]).convert("RGB")
        h = imagehash.phash(im, hash_size=8) # 64-bit DCT-based perceptual hash
        hashes.append(h)
        splits.append(d["split"])
        paths.append(d["path"])
    except Exception:
        pass

print(f"Hashed {len(hashes)} images.")
splits = np.array(splits)
n = len(hashes)

# ----- Pairwise Hamming distance -----
# imagehash hashes support subtraction directly -> Hamming distance
print("\nComputing pairwise Hamming distances (this takes a minute for ~542 images)...")
THRESHOLDS = [3, 5, 8, 10]
counts = {t: {"dup": 0, "leak": 0} for t in THRESHOLDS}

for i in range(n):
    for j in range(i + 1, n):
        dist = hashes[i] - hashes[j] # Hamming distance, 0-64
        for t in THRESHOLDS:
            if dist <= t:
                counts[t]["dup"] += 1
                if splits[i] != splits[j]:
                    counts[t]["leak"] += 1

print(f"\n{'Hamming threshold':>18} | {'Near-dup pairs':>15} | {'Leakage pairs':>14}")
print("-" * 54)
```

```

for t in THRESHOLDS:
    marker = " <-- suggested default" if t == 5 else ""
    print(f"{t:>18d} | {counts[t]['dup']:>15d} | {counts[t]['leak']:>14d}{marker}")

print("\nUse the row that best matches a 'near duplicate' by eye, or keep the")
print("suggested default (Hamming distance <= 5 out of 64 bits). Report the")
print("(dup, leak) numbers for whichever threshold you pick as your Table 1")
print("perceptual-hash baseline, and state 'DCT-based pHash (64-bit), Hamming")
print("distance <= [X]' in Section 2.2.2.")

```

Loaded 542 original images (435 train, 107 val).
Hashed 542 images.

Computing pairwise Hamming distances (this takes a minute for ~542 images)...

Hamming threshold	Near-dup pairs	Leakage pairs
3	25	13
5	29	13 <-- suggested default
8	37	16
10	40	17

Use the row that best matches a 'near duplicate' by eye, or keep the suggested default (Hamming distance <= 5 out of 64 bits). Report the (dup, leak) numbers for whichever threshold you pick as your Table 1 perceptual-hash baseline, and state 'DCT-based pHash (64-bit), Hamming distance <= [X]' in Section 2.2.2.

```

# =====
# GRAD-CAM FIGURE (fills Figure 7, Section 3.4)
#
# Trains CyberNetX once (seed 42, same architecture as everywhere
# else), then immediately runs Grad-CAM on a few stock test images
# and a few real Hateful Memes images using that same model, and
# saves the comparison figure. Self-contained so you don't need to
# save/reload a checkpoint across sessions.
#
# Requires: the 330/71/71 clean folder (already fixed) and the
# Hateful Memes data (downloads automatically if missing, same as
# complete_pipeline.py).
# =====
import os, json, random, subprocess, sys
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, Model, Input
from tensorflow.keras.applications import ResNet50, InceptionResNetV2, Xception
from tensorflow.keras.applications.resnet50 import preprocess_input as pre_res
from tensorflow.keras.applications.inception_resnet_v2 import preprocess_input as pre_inc
from tensorflow.keras.applications.xception import preprocess_input as pre_xcep
import matplotlib.pyplot as plt
import matplotlib.cm as cm

from google.colab import drive
drive.mount('/content/drive', force_remount=True)

# ----- 1. Verify clean data (same hard check as before) -----
clean = "/content/drive/MyDrive/cyberbullying_dataset_clean/"
counts = {}
for split in ["train", "val", "test"]:
    n = 0
    for cls in ["bully", "no_bully"]:
        d = os.path.join(clean, split, cls)
        n += len(os.listdir(d)) if os.path.isdir(d) else 0
    counts[split] = n
expected = {"train": 330, "val": 71, "test": 71}
if counts != expected:
    raise RuntimeError(f"STOP: clean dataset counts are {counts}, expected {expected}.")
print("Data check passed - 330/71/71 confirmed.\n")

# ----- 2. Get Hateful Memes (skips if already present) -----
hm_root = "/content/hateful_memes/hateful_memes"
if not os.path.exists(hm_root):
    print("Downloading Hateful Memes...")
    os.makedirs("/root/.kaggle", exist_ok=True)
    drive_key = "/content/drive/MyDrive/kaggle.json"
    if os.path.exists(drive_key):
        os.system(f"cp '{drive_key}' /root/.kaggle/kaggle.json")
    os.system("chmod 600 /root/.kaggle/kaggle.json")
    subprocess.run([sys.executable, "-m", "pip", "install", "-q", "kaggle"])
    os.system("kaggle datasets download -d williamberrios/hateful-memes -p /content")
    os.system("unzip -q /content/hateful-memes.zip -d /content/hateful_memes")
    print("Hateful Memes ready:", os.path.exists(hm_root))

# ----- 3. Determinism + training (seed 42, same as before) -----
def set_full_determinism(seed):
    os.environ["PYTHONHASHSEED"] = str(seed)
    os.environ["TF_DETERMINISTIC_OPS"] = "1"
    os.environ["TF_CUDNN_DETERMINISTIC"] = "1"
    random.seed(seed); np.random.seed(seed); tf.random.set_seed(seed)
    try:
        tf.config.experimental.enable_op_determinism()
    except Exception:
        pass

```

```

SEED = 42
set_full_determinism(SEED)
IMG, BATCH = (224, 224), 32

def load_split(split, shuffle):
    return tf.keras.utils.image_dataset_from_directory(
        clean + split, image_size=IMG, batch_size=BATCH,
        label_mode="int", class_names=["bully", "no_bully"],
        shuffle=shuffle, seed=SEED if shuffle else None)

train_ds = load_split("train", True).prefetch(tf.data.AUTOTUNE)
val_ds = load_split("val", False).prefetch(tf.data.AUTOTUNE)

# Names kept explicit so Grad-CAM can find the Xception sub-model later
XCEPTION_NAME = "xception"

def build_model():
    aug = tf.keras.Sequential([layers.RandomFlip("horizontal"),
                               layers.RandomRotation(0.1), layers.RandomZoom(0.1)])

    def bb(net, name):
        m = net(include_top=False, pooling="avg", weights="imagenet")
        m.trainable = False; m._name = name
        return m

    res, inc, xcp = bb(ResNet50, "resnet50"), bb(InceptionResNetV2, "incresv2"), bb(Xception, XCEPTION_NAME)
    inp = Input((224, 224, 3)); a = aug(inp)
    fr = res(layers.Lambda(pre_res)(a))
    fi = inc(layers.Lambda(pre_inc)(a))
    fx = xcp(layers.Lambda(pre_xcp)(a))
    z = layers.Concatenate()([fr, fi, fx])
    z = layers.BatchNormalization()(z)
    z = layers.Dense(512, activation="relu")(z)
    z = layers.Dropout(0.5)(z)
    z = layers.Dense(256, activation="relu")(z)
    z = layers.Dropout(0.5)(z)
    out = layers.Dense(1, activation="sigmoid")(z)
    model = Model(inp, out)
    model.compile(optimizer=tf.keras.optimizers.Adam(1e-4), loss="binary_crossentropy", metrics=["accuracy"])
    return model

print("\nTraining CyberNetX (seed 42)...")
model = build_model()
cb = [tf.keras.callbacks.EarlyStopping(patience=8, restore_best_weights=True, monitor="val_loss")]
model.fit(train_ds, validation_data=val_ds, epochs=40, callbacks=cb, verbose=1)
print("Training done.\n")

# ----- 4. Pick sample images -----
def first_n_paths(folder, n):
    paths = []
    for cls in ["bully", "no_bully"]:
        d = os.path.join(clean, "test", cls)
        for f in sorted(os.listdir(d))[:n]:
            paths.append(os.path.join(d, f))
    return paths[:n]

stock_paths = first_n_paths(clean, 3)

with open(os.path.join(hm_root, "test_seen.jsonl")) as f:
    meme_recs = [json.loads(l) for l in f][:3]
meme_paths = [os.path.join(hm_root, r["img"]) for r in meme_recs]

print("Stock sample images:", stock_paths)
print("Meme sample images:", meme_paths)

# ----- 5. Grad-CAM -----
def get_last_conv_layer_name(sub_model):
    for layer in reversed(sub_model.layers):
        if len(layer.output_shape) == 4:
            return layer.name
    raise ValueError("No conv layer found.")

def make_gradcam_heatmap(img_array, xception_submodel, last_conv_layer_name):
    grad_model = tf.keras.models.Model(
        xception_submodel.inputs,
        [xception_submodel.get_layer(last_conv_layer_name).output, xception_submodel.output])
    with tf.GradientTape() as tape:
        conv_outputs, predictions = grad_model(img_array)
        loss = tf.reduce_mean(predictions)
    grads = tape.gradient(loss, conv_outputs)
    pooled_grads = tf.reduce_mean(grads, axis=(0, 1, 2))
    conv_outputs = conv_outputs[0]
    heatmap = conv_outputs @ pooled_grads[..., tf.newaxis]
    heatmap = tf.squeeze(heatmap)
    heatmap = tf.maximum(heatmap, 0) / (tf.math.reduce_max(heatmap) + 1e-8)
    return heatmap.numpy()

def overlay_heatmap(img_path, heatmap, alpha=0.45, img_size=(224, 224)):
    img = tf.keras.utils.load_img(img_path, target_size=img_size)
    img = tf.keras.utils.img_to_array(img)
    heatmap_resized = np.uint8(255 * heatmap)
    jet = cm.get_cmap("jet")
    iet_colors = iet(np.arange(256))[: :3]

```

```

jet_heatmap = jet_colors[heatmap_resized]
jet_heatmap = tf.keras.utils.array_to_img(jet_heatmap)
jet_heatmap = jet_heatmap.resize((img_size[1], img_size[0]))
jet_heatmap = tf.keras.utils.img_to_array(jet_heatmap)
overlaid = jet_heatmap * alpha + img
return tf.keras.utils.array_to_img(overlaid)

xception_submodel = model.get_layer(XCEPTION_NAME)
last_conv = get_last_conv_layer_name(xception_submodel)

all_paths = [("Stock", p) for p in stock_paths] + [("Meme", p) for p in meme_paths]
n = len(all_paths)
fig, axes = plt.subplots(2, n, figsize=(3 * n, 6.2))

for col, (label, path) in enumerate(all_paths):
    img = tf.keras.utils.load_img(path, target_size=(224, 224))
    arr = np.expand_dims(tf.keras.utils.img_to_array(img), axis=0)
    pre = pre_xcep(arr.copy())
    prob = float(model.predict(arr, verbose=0)[0][0])
    pred_label = "no_bully/not-harmful" if prob >= 0.5 else "bully/harmful"

    heatmap = make_gradcam_heatmap(pre, xception_submodel, last_conv)
    overlay = overlay_heatmap(path, heatmap)

    axes[0, col].imshow(img)
    axes[0, col].set_title(f"{label} (original)\npred: {pred_label}", fontsize=8.5)
    axes[0, col].axis("off")

    axes[1, col].imshow(overlay)
    axes[1, col].set_title(f"{label} (Grad-CAM)", fontsize=8.5)
    axes[1, col].axis("off")

plt.tight_layout()
out_path = "/content/figure7_gradcam.png"
plt.savefig(out_path, dpi=200, bbox_inches="tight")
print(f"\nSaved to {out_path} - download it (left sidebar > Files) and send it back to Claude.")

```



```

def make_gradcam_heatmap(img_array, xception_submodel, last_conv_layer_name):
    grad_model = tf.keras.models.Model(
        xception_submodel.inputs,
        [xception_submodel.get_layer(last_conv_layer_name).output, xception_submodel.output])
    with tf.GradientTape() as tape:
        conv_outputs, predictions = grad_model(img_array)
        loss = tf.reduce_mean(predictions)
    grads = tape.gradient(loss, conv_outputs)
    pooled_grads = tf.reduce_mean(grads, axis=(0, 1, 2))
    conv_outputs = conv_outputs[0]
    heatmap = conv_outputs @ pooled_grads[..., tf.newaxis]
    heatmap = tf.squeeze(heatmap)
    heatmap = tf.maximum(heatmap, 0) / (tf.math.reduce_max(heatmap) + 1e-8)
    return heatmap.numpy()

def overlay_heatmap(img_path, heatmap, alpha=0.45, img_size=(224, 224)):
    img = tf.keras.utils.load_img(img_path, target_size=img_size)
    img = tf.keras.utils.img_to_array(img)
    heatmap_resized = np.uint8(255 * heatmap)
    jet = cm.get_cmap("jet")
    jet_colors = jet(np.arange(256))[:, :3]
    jet_heatmap = jet_colors[heatmap_resized]
    jet_heatmap = tf.keras.utils.array_to_img(jet_heatmap)
    jet_heatmap = jet_heatmap.resize((img_size[1], img_size[0]))
    jet_heatmap = tf.keras.utils.img_to_array(jet_heatmap)
    overlaid = jet_heatmap * alpha + img
    return tf.keras.utils.array_to_img(overlaid)

XCEPTION_NAME = "xception"
xception_submodel = model.get_layer(XCEPTION_NAME)
last_conv = get_last_conv_layer_name(xception_submodel)
print("Using last conv layer:", last_conv)

all_paths = [("Stock", p) for p in stock_paths] + [("Meme", p) for p in meme_paths]
n = len(all_paths)
fig, axes = plt.subplots(2, n, figsize=(3 * n, 6.2))

for col, (label, path) in enumerate(all_paths):
    img = tf.keras.utils.load_img(path, target_size=(224, 224))
    arr = np.expand_dims(tf.keras.utils.img_to_array(img), axis=0)
    pre = pre_xcep(arr.copy())
    prob = float(model.predict(arr, verbose=0)[0][0])
    pred_label = "no_bully/not-harmful" if prob >= 0.5 else "bully/harmful"

    heatmap = make_gradcam_heatmap(pre, xception_submodel, last_conv)
    overlay = overlay_heatmap(path, heatmap)

```